



KT AIVLE School

클라우드 현대화 이해와 주요 서비스



Contents

Lab 01. 환경 준비

Lab 02. AWS CodePipeline Basics

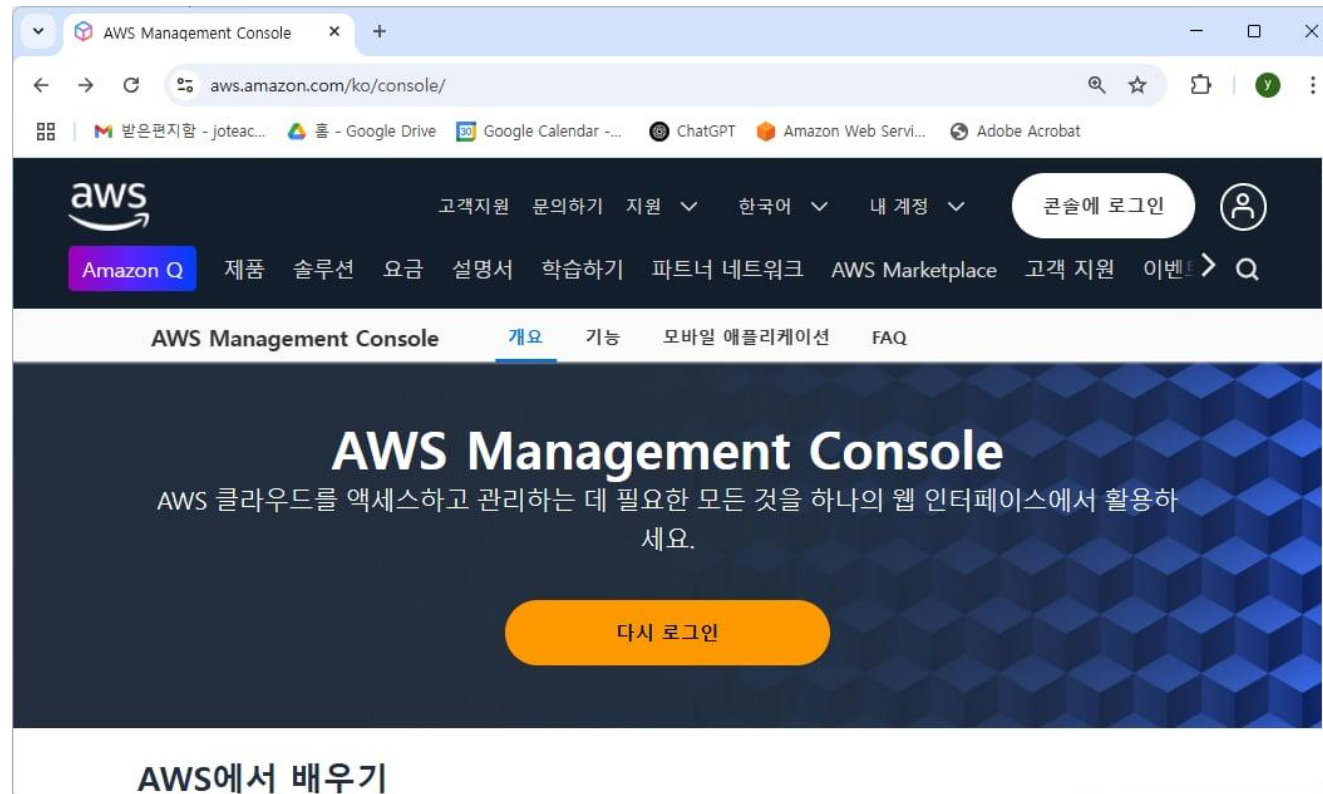
Lab 03. Amazon EC2 Instance 생성 및 연결

Lab 01.

환경 준비

Step 1. AWS 로그인 사이트 접속

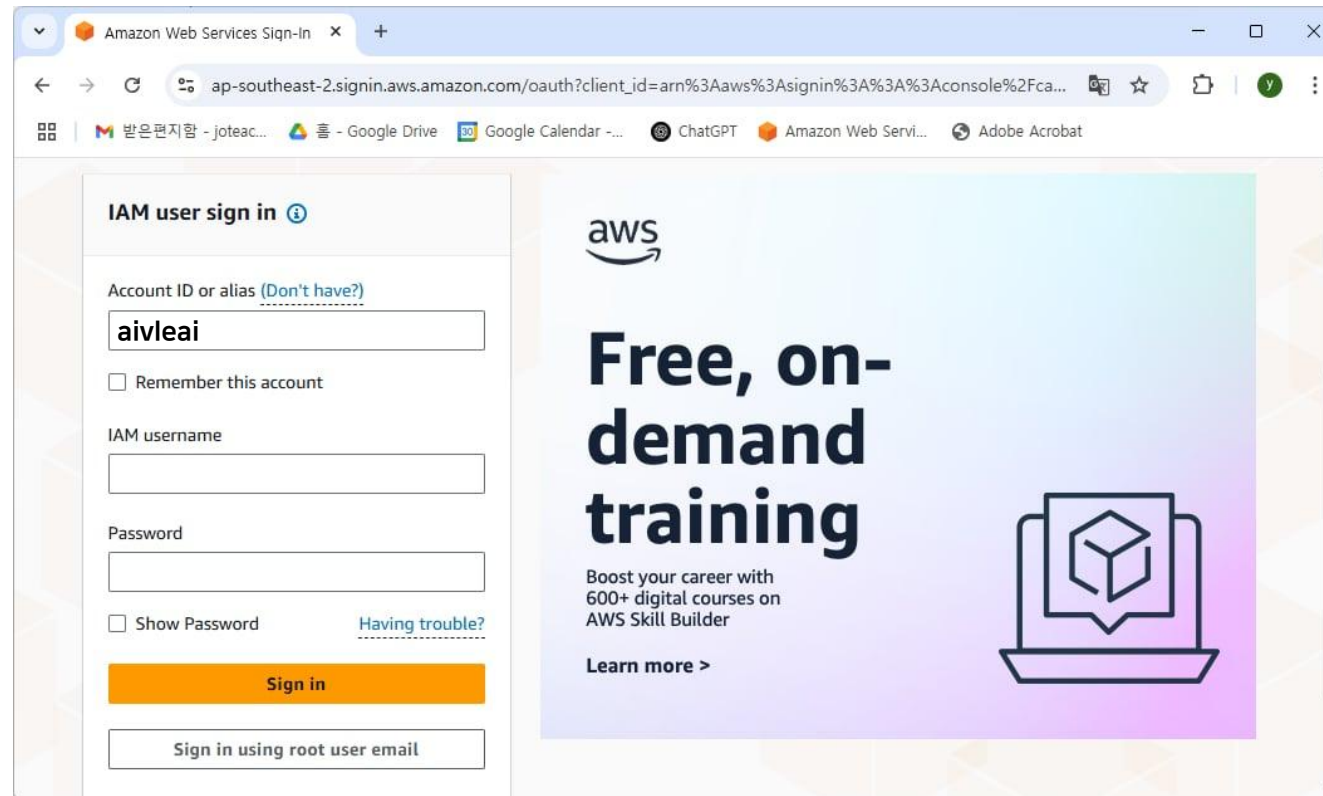
제공된 링크(<https://aws.amazon.com/ko/console/>)로 접속해 "콘솔에 로그인"을 클릭합니다.



Step 2. 계정 로그인

제공받은 IAM username과 Password를 입력하고 Sign in을 클릭합니다.

(Account ID: **aivleai**)



The screenshot shows the AWS IAM user sign-in page in a web browser. The browser tab is titled "Amazon Web Services Sign-In". The address bar shows the URL: `ap-southeast-2.signin.aws.amazon.com/oauth?client_id=arn%3Aaws%3Asign%3A%3A%3Aconsole%2Fca...`. The page has a light blue and white background. On the left, there is a form titled "IAM user sign in" with a help icon. The form contains the following fields and options:

- Account ID or alias ([Don't have?](#)): A text input field containing the value "aivleai".
- ☐ Remember this account
- IAM username: A text input field.
- Password: A text input field.
- ☐ Show Password [Having trouble?](#)
- (orange button)
- (white button)

On the right side of the page, there is a large promotional banner for "Free, on-demand training". The banner features the AWS logo, the text "Free, on-demand training", and a sub-headline "Boost your career with 600+ digital courses on AWS Skill Builder". Below the text is a "Learn more >" link. To the right of the text is an illustration of a laptop with a cube icon on its screen.

Step 3. 리전(Region) 확인

오른쪽 상단에서 선택된 리전을 확인할 수 있습니다. **안내받은 리전으로 선택합니다.**

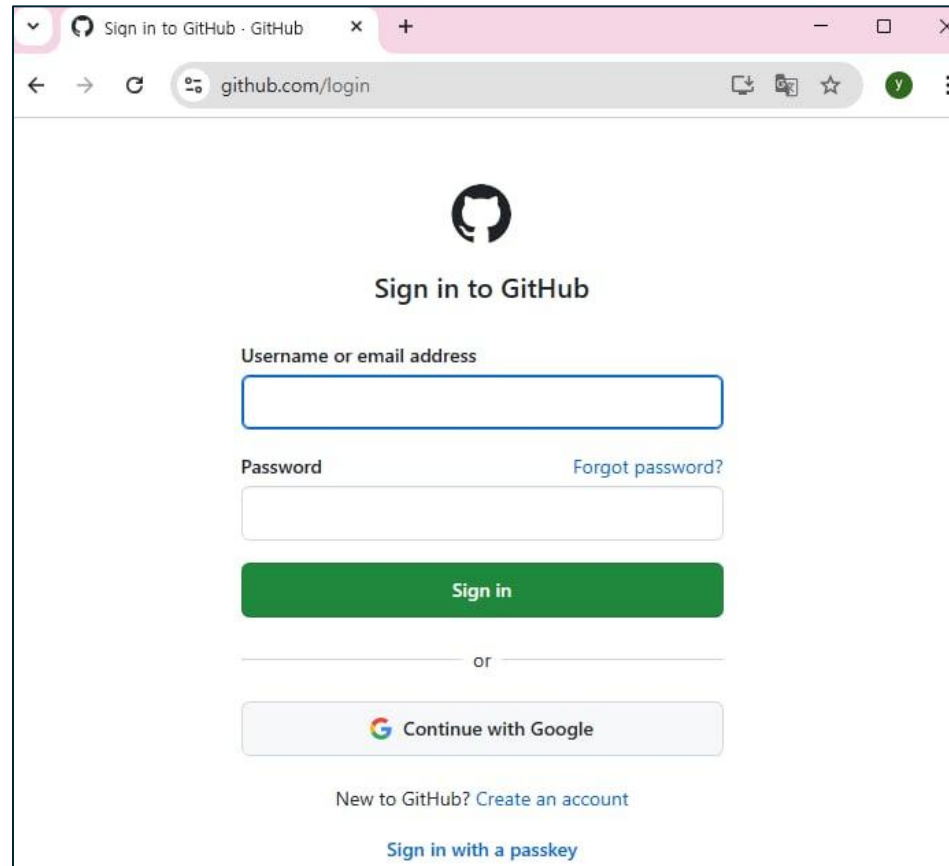


The screenshot shows the AWS Management Console interface with the region dropdown menu open. The menu is divided into two sections: '미국' (US) and '아시아 태평양' (Asia Pacific). The '서울' (Seoul) option under the '아시아 태평양' section is highlighted in blue, indicating it is the selected region.

미국	
버지니아 북부	us-east-1
오하이오	us-east-2
캘리포니아	us-west-1
오레곤	us-west-2
아시아 태평양	
뭄바이	ap-south-1
오사카	ap-northeast-3
서울	ap-northeast-2
싱가포르	ap-southeast-1
시드니	ap-southeast-2
도쿄	ap-northeast-1

Step 1. 깃허브 로그인

깃허브 사이트(<https://github.com/>)에 접속하고, 로그인합니다.

A screenshot of a web browser showing the GitHub login page. The browser's address bar displays 'github.com/login'. The page features the GitHub logo at the top, followed by the text 'Sign in to GitHub'. Below this, there are two input fields: 'Username or email address' and 'Password'. To the right of the password field is a link that says 'Forgot password?'. A green 'Sign in' button is positioned below the input fields. Underneath the button is a horizontal line with the word 'or' in the center. Below this line is a button labeled 'Continue with Google' with the Google logo. At the bottom of the page, there are two links: 'New to GitHub? Create an account' and 'Sign in with a passkey'.

Step 2. Repository 생성

새로운 repository를 생성합니다. repository의 이름을 작성하고, Private으로 생성합니다.

Create a new repository

Preview

Switch back to classic experience

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *

joteacher

Repository name *

test-website-repo

test-website-repo is available.

Great repository names are short and memorable. How about **didactic-waffle**?

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Private

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

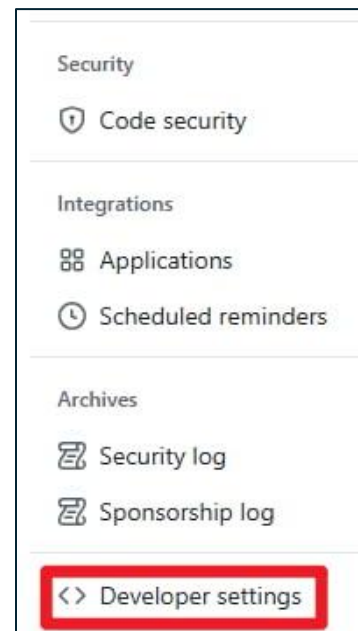
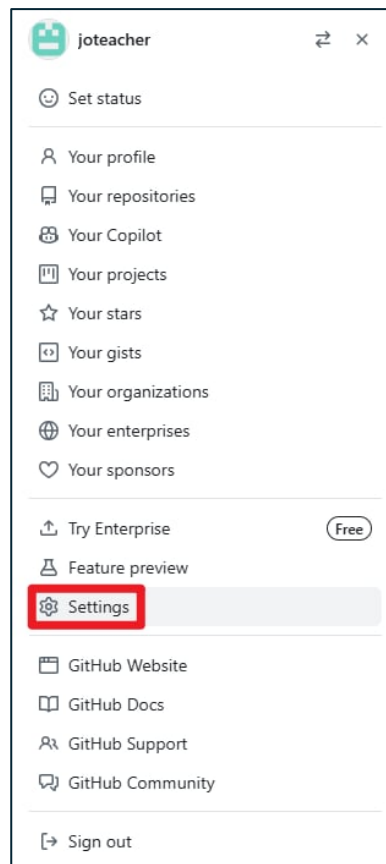
No license

Create repository

Step 3. Access token 생성

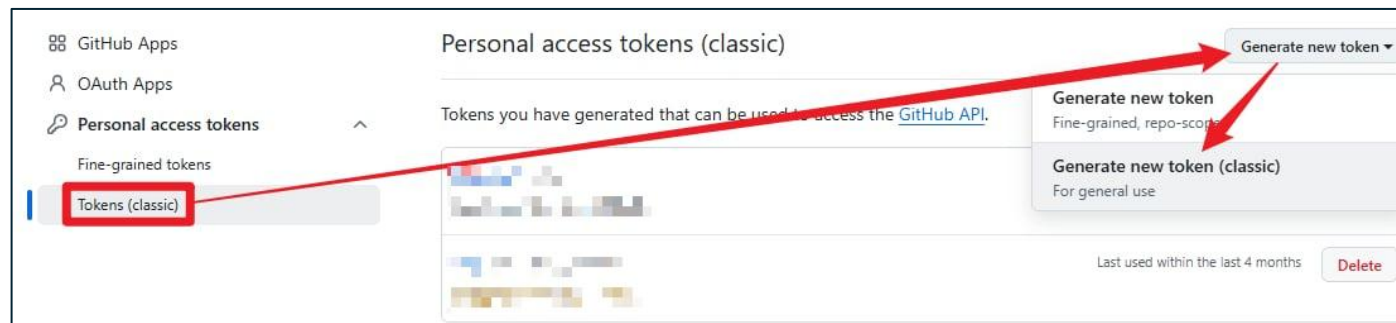
GitHub 우측 상단 프로필 아이콘을 클릭하고, 하단의 "setting"를 클릭합니다.

좌측 하단에 "Developer settings"를 클릭합니다.



Step 3. Access token 생성

Personal access tokens에서 "Tokens (classic)" > "Generate new token" > "Generate new token (classic)"을 클릭합니다.



Note와 Expiration을 설정합니다.

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

What's this token for?

Expiration

The token will expire on the selected date

Step 3. Access token 생성

Select scopes에서 "repo"를 선택하고,
"Generate token"을 클릭해 토큰을 생성합니다.

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

☒ repo	Full control of private repositories
☐ repo:status	Access commit status
☐ repo_deployment	Access deployment status
☐ public_repo	Access public repositories
☐ repo:invite	Access repository invitations
☐ security_events	Read and write security events

생성된 access token을 확인하고, 로컬에 저장합니다.

Personal access tokens (classic) Generate new token ▾

Tokens you have generated that can be used to access the [GitHub API](#).

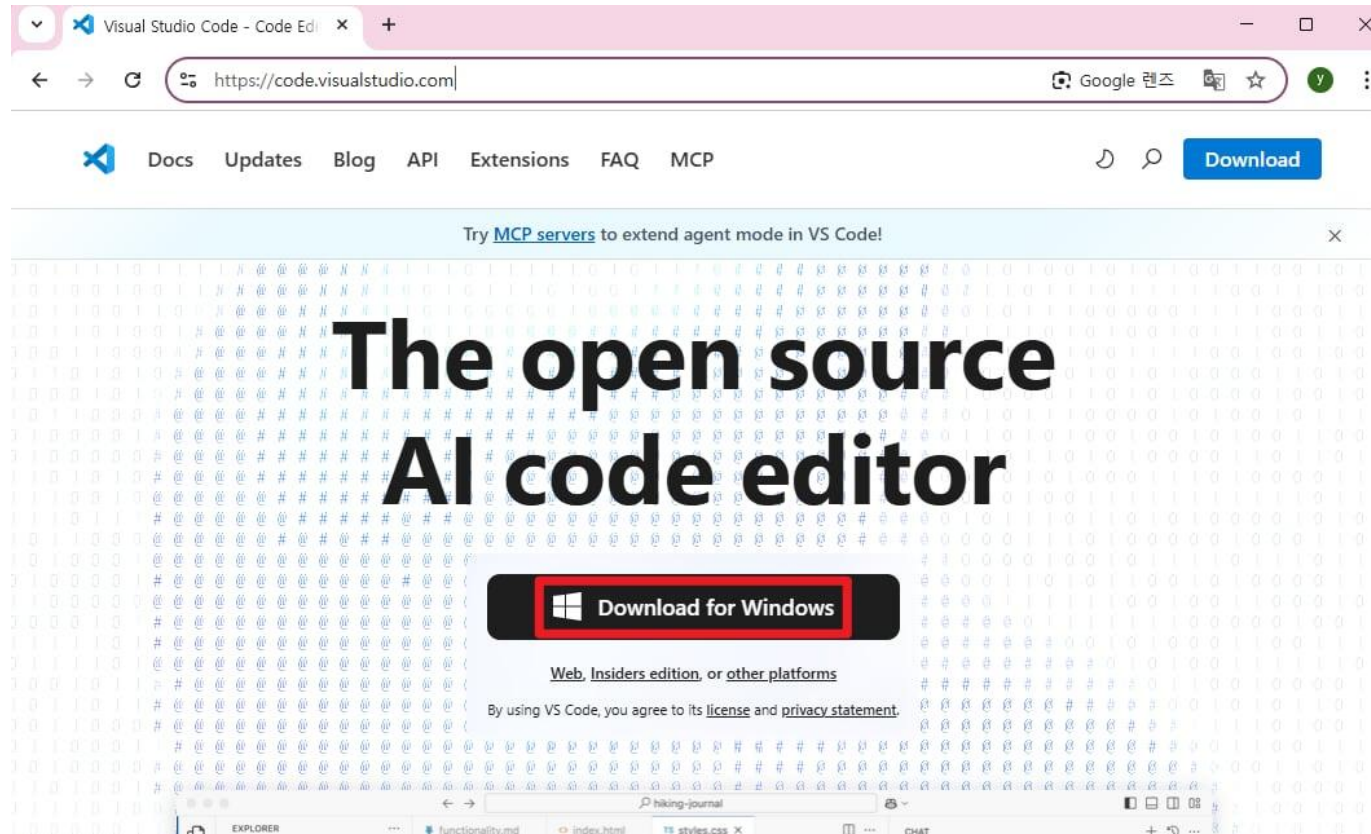
 Make sure to copy your personal access token now. You won't be able to see it again!

☒ g [redacted]		Delete
[redacted]	Never used	Delete
[redacted]	Last used within the last 4 months	Delete

VS Code 및 Git 설치

Step 1. VS Code 설치

VS Code를 설치합니다. (<https://code.visualstudio.com/>)

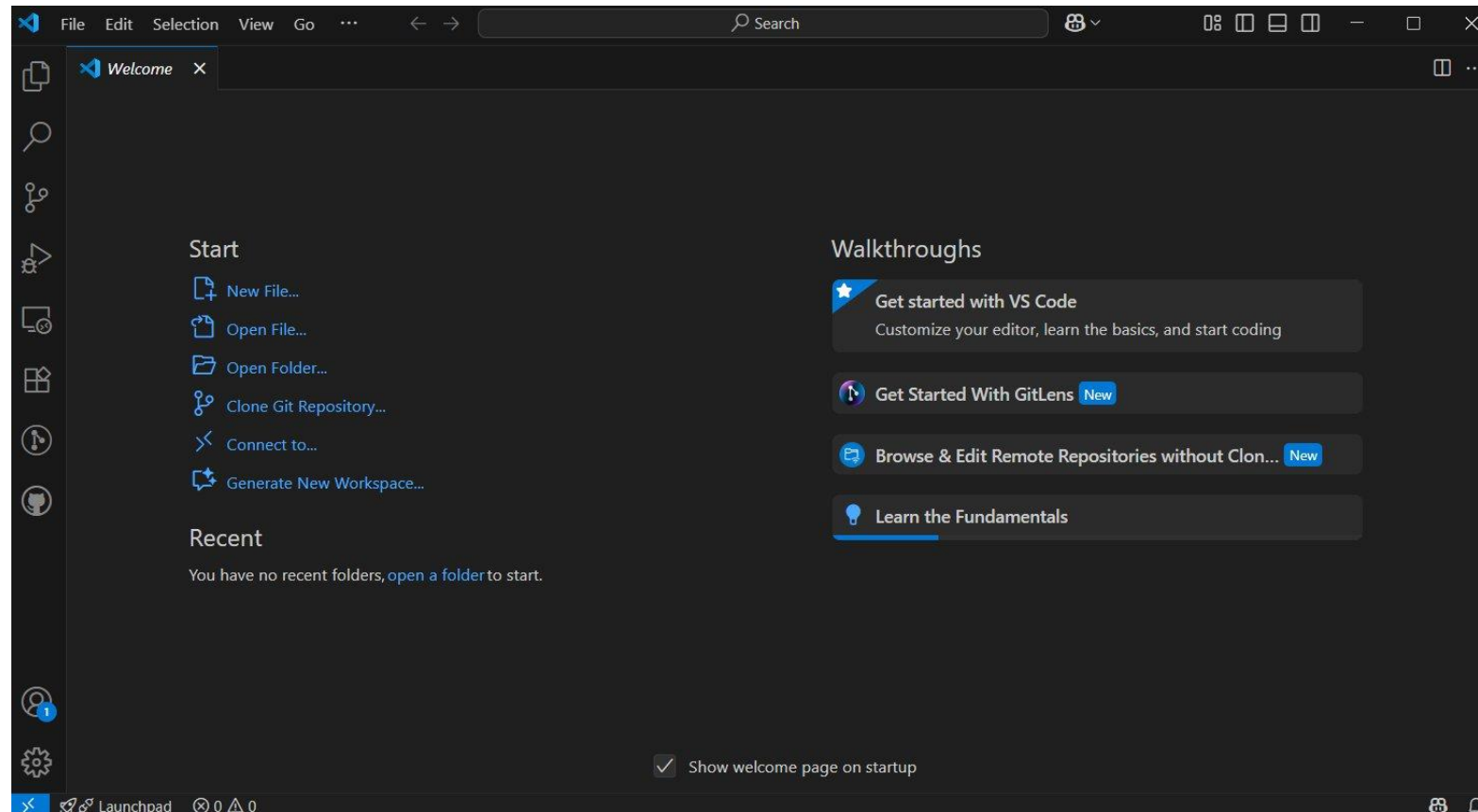


VS Code 및 Git 설치



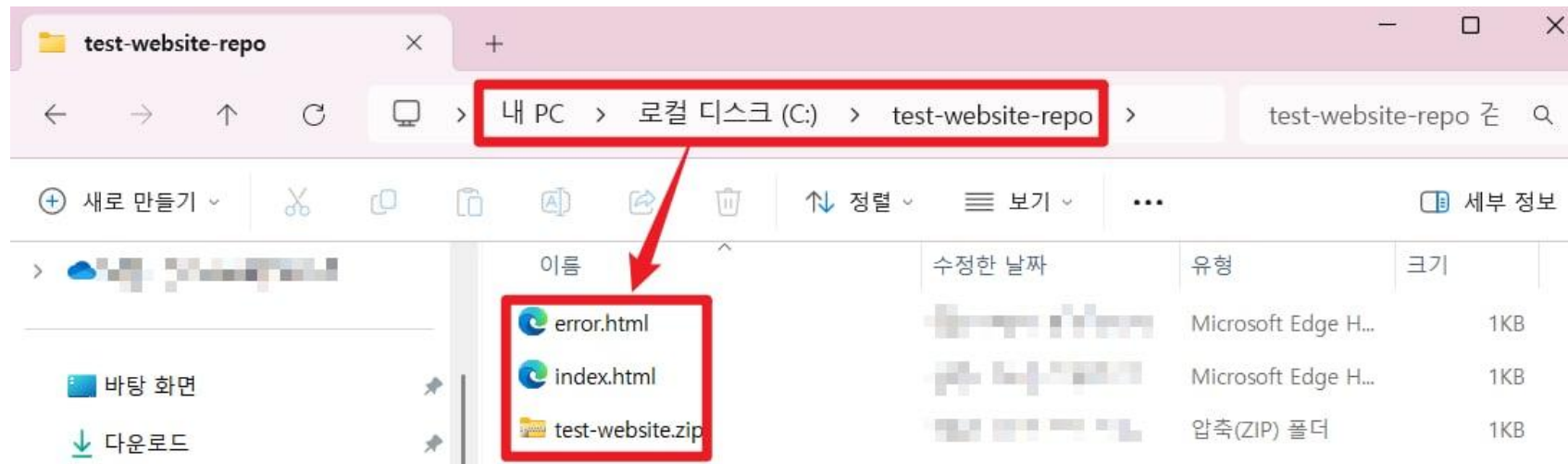
Step 1. VS Code 설치

설치한 VS Code를 실행합니다.



Step 2. 파일 준비

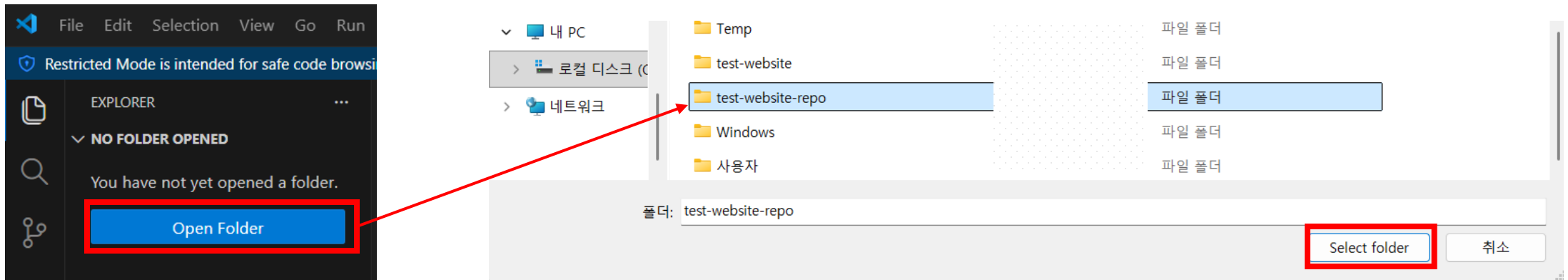
제공된 파일을 원하는 경로에 이동시킵니다.



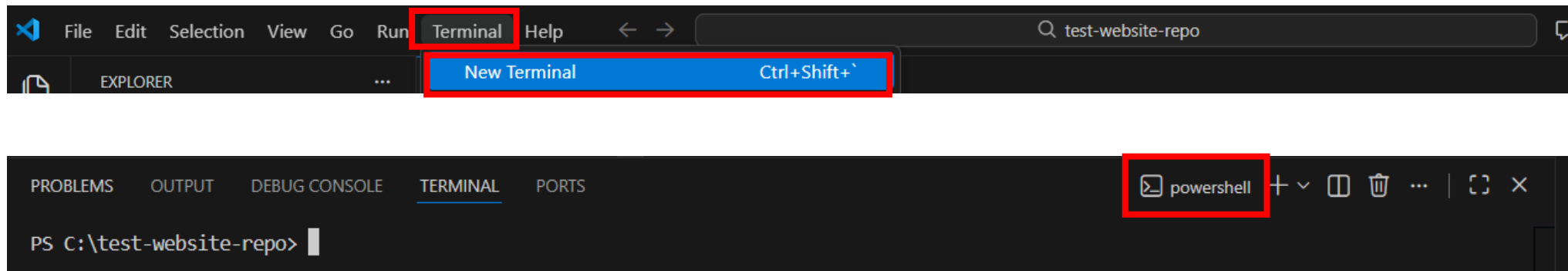
VS Code 및 Git 설치

Step 2. 파일 준비

제공된 파일을 옮긴 폴더를 찾아서 VS Code에서 엽니다.

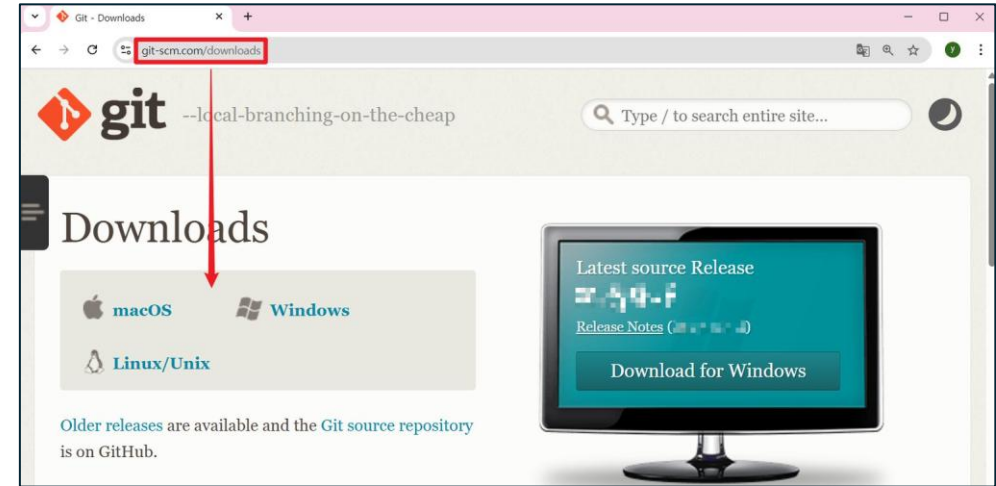


VS코드의 터미널이 정상적으로 열리는 지 확인합니다.



Step 3. Git 설치

사이트(<https://git-scm.com/downloads>)에서 설치 파일을 다운로드 받습니다.(이미 설치되었다면 하지 않아도 됩니다.)



VS코드의 터미널에서 git 설치가 잘 되었는지 확인합니다.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\test-website-repo> git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
          [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
          [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
          [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
          [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
          <command> [<args>]
```


Step 4. Git 연동

Git을 초기화 합니다.

```
git init
```

```
PS C:\test-website-repo> git init  
Initialized empty Git repository in C:/test-website-repo/.git/
```

Git 인증 정보를 등록합니다.

```
git config --global user.name <github 계정명>  
git config --global user.email <github 이메일주소>
```

```
PS C:\test-website-repo> git config --global user.name [redacted]  
PS C:\test-website-repo> git config --global user.email [redacted]  
PS C:\test-website-repo>
```

VS Code 및 Git 설치

Step 4. Git 연동

Git 원격 연결합니다.

```
git remote add origin https://github.com/<계정명>/test-website-repo.git
```

```
PS C:\test-website-repo> git remote add origin https://github.com/[REDACTED]/test-website-repo.git
PS C:\test-website-repo>
```

현재 디렉터리에 있는 파일을 stage에 올립니다.

```
git add .
```

```
PS C:\test-website-repo> git add .
```

VS Code 및 Git 설치

Step 4. Git 연동

Staging Area에 올라간 파일을 Commit 합니다.

```
git commit -m "commit 1"
```

```
PS C:\test-website-repo> git commit -m "commit 1"
[master (root-commit) 00cda7] commit 1
3 files changed, 22 insertions(+)
create mode 100644 error.html
create mode 100644 index.html
create mode 100644 test-website.zip
```

Step 4. Git 연동

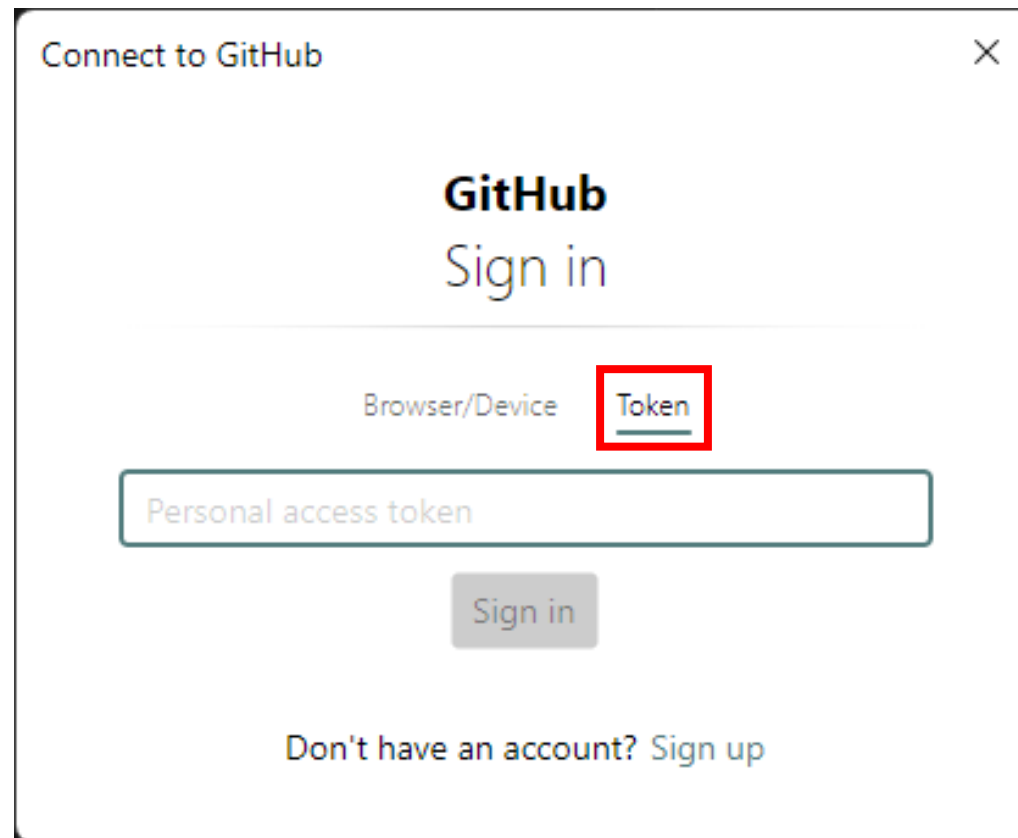
Commit한 파일을 Push해서 원격 저장소로 보냅니다.

```
git branch -M main  
git push -u origin main
```

```
PS C:\test-website-repo> git branch -M main  
PS C:\test-website-repo> git push -u origin main  
Enumerating objects: 5, done.  
Counting objects: 100% (5/5), done.  
Delta compression using up to 18 threads  
Compressing objects: 100% (5/5), done.  
Writing objects: 100% (5/5), 1.31 KiB | 1.31 MiB/s, done.  
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)  
remote: Resolving deltas: 100% (1/1), done.  
To https://github.com/[redacted]/test-website-repo.git  
* [new branch]      main -> main  
branch 'main' set up to track 'origin/main'.
```

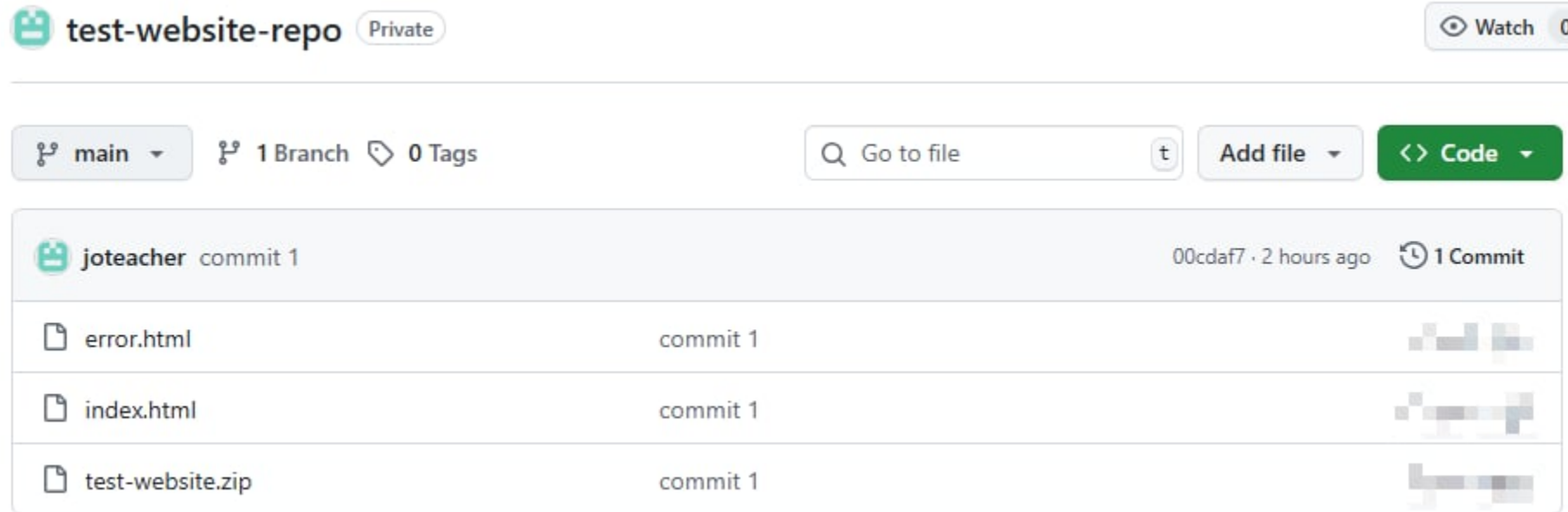
Step 4. Git 연동

Connection to GitHub 팝업이 뜬다면, Token을 클릭 후 로컬에 저장한 **Access Token** 값을 입력합니다.

A screenshot of the 'Connect to GitHub' dialog box. The dialog has a title bar with 'Connect to GitHub' and a close button. The main content area displays the GitHub logo and 'Sign in'. Below this, there are two options: 'Browser/Device' and 'Token'. The 'Token' option is highlighted with a red rectangular box. Under the 'Token' option, there is a text input field with the placeholder text 'Personal access token'. Below the input field is a 'Sign in' button. At the bottom of the dialog, there is a link that says 'Don't have an account? Sign up'.

Step 4. Git 연동

GitHub에서 파일이 push 되었는지 확인합니다.



The screenshot shows a GitHub repository page for 'test-website-repo' (Private). The repository is owned by 'joteacher' and has 1 commit. The files listed are 'error.html', 'index.html', and 'test-website.zip', all committed in 'commit 1'. The commit hash is '00cdaf7' and it was made '2 hours ago'.

test-website-repo Private Watch 0

main 1 Branch 0 Tags Go to file Add file Code

joteacher commit 1 00cdaf7 · 2 hours ago 1 Commit

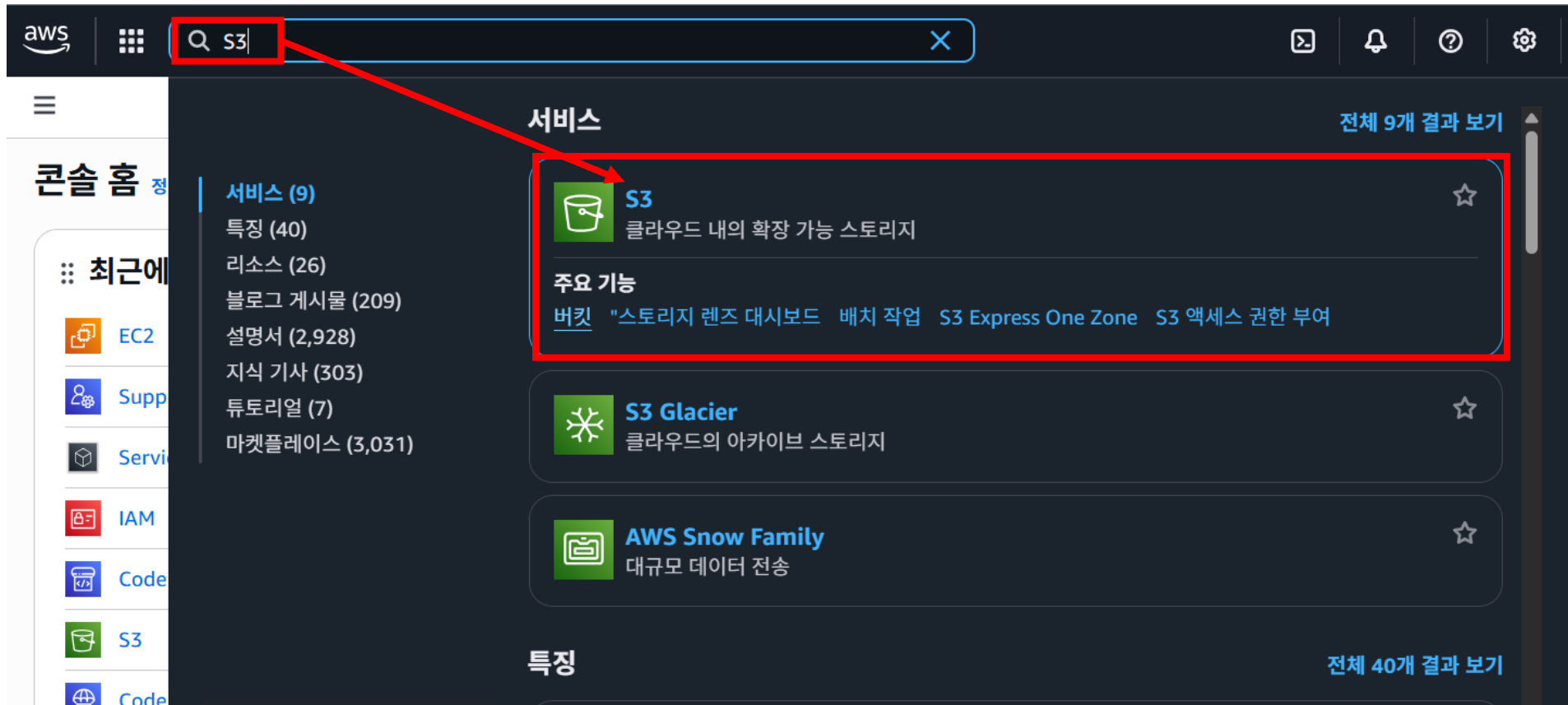
error.html	commit 1	
index.html	commit 1	
test-website.zip	commit 1	

Lab 02.

AWS CodePipeline Basics

Step 1. Source Bucket 생성

AWS 콘솔에서 S3 서비스를 클릭합니다.



Step 1. Source Bucket 생성

범용 버킷에서 "버킷 만들기"를 클릭합니다.

Amazon S3

버킷

범용 버킷

디렉터리 버킷

테이블 버킷

벡터 버킷

액세스 관리 및 보안

액세스 지점

Access Grants

IAM Access Analyzer

스토리지 관리 및 인사이트

Storage Lens

배치 작업

범용 버킷

모든 AWS 리전

디렉터리 버킷

범용 버킷 (12) 정보

버킷은 S3에 저장되는 데이터의 컨테이너입니다.

이름으로 버킷 찾기

버킷 만들기

이름	AWS 리전	생성 날짜

Step 1. Source Bucket 생성

Bucket의 이름은 "user##-source-website"로 합니다. (다른 이름으로 변경해도 됩니다.)

일반 구성

AWS 리전

아시아 태평양(서울) ap-northeast-2

버킷 이름 정보

user00-source-website

버킷 이름은 3~63자여야 하며 글로벌 네임스페이스 내에서 고유해야 합니다. 또한 버킷 이름은 문자나 숫자로 시작하고 끝나야 합니다. 유효한 문자는 a-z, 0-9, hyphen, period, underscore, space, and dash.

기존 버킷에서 설정 복사 - 선택 사항

다음 구성의 버킷 설정만 복사됩니다.

버킷 선택

형식: s3://bucket/prefix

Step 1. Source Bucket 생성

버킷 버전 관리를 "활성화"로 체크 합니다.

버킷 버전 관리

버전 관리는 객체의 여러 버전을 동일한 버킷에서 관리하기 위한 수단입니다. 버전 관리를 사용하여 Amazon S3 버킷에 저장된 모든 객체와 애플리케이션 장애를 모두 복구할 수 있습니다. [자세히 알아보기](#)

버킷 버전 관리

☐ 비활성화

☒ 활성화

맨 아래 "버킷 만들기"를 클릭해 Source용 버킷을 생성하고, 생성된 버킷을 확인합니다.

범용 버킷 (1) 정보

버킷은 S3에 저장되는 데이터의 컨테이너입니다.

이름	AWS 리전	생성 날짜
☐ user00-source-website	아시아 태평양(서울) ap-northeast-2	

Step 2. Prod Bucket 생성

두 번째 버킷인 Prod용 버킷을 생성합니다.

일반 구성

AWS 리전
아시아 태평양(서울) ap-northeast-2

버킷 이름 | 정보

버킷 이름은 3~63자여야 하며 글로벌 네임스페이스 내에서 고유해야 합니다. 또한 버킷 이름은 문자나 숫자로 시작하고 끝나야 합니다. 유효한 문자는 a-z,

기존 버킷에서 설정 복사 - 선택 사항

다음 구성의 버킷 설정만 복사됩니다.

버킷 선택

형식: s3://bucket/prefix

Step 2. Prod Bucket 생성

객체 소유권에서 "ACL 활성화됨"을 체크합니다.

객체 소유권 정보

다른 AWS 계정에서 이 버킷에 작성한 객체의 소유권 및 액세스 제어 목록(ACL)의 사용을 제어합니다. 객체 소유권은 객체에 대한 액세스를 지정할 수 있는 사용자를 결정합니다.

☐ ACL 비활성화됨(권장)

이 버킷의 모든 객체는 이 계정이 소유합니다. 이 버킷과 그 객체에 대한 액세스는 정책을 통해서만 지정됩니다.

☒ ACL 활성화됨

이 버킷의 객체는 다른 AWS 계정에서 소유할 수 있습니다. 이 버킷 및 객체에 대한 액세스는 ACL을 사용하여 지정할 수 있습니다.

 각 객체의 액세스를 개별적으로 제어하거나 객체 라이터가 업로드하는 데이터를 소유하도록 해야 하는 경우가 아니라면 ACL을 비활성화하는 것이 좋습니다. ACL 대신 버킷 정책을 사용하여 계정 외부의 사용자와 데이터를 공유하면 권한을 관리하고 감사하기가 용이합니다.

Step 2. Prod Bucket 생성

S3는 기본적으로 퍼블릭 액세스 차단 설정이 되어 있습니다. "모든 퍼블릭 액세스 차단"을 체크 해제하고, 아래쪽에 퍼블릭 상태가 될 수 있음에 체크 합니다.

이 버킷의 퍼블릭 액세스 차단 설정

퍼블릭 액세스는 ACL(액세스 제어 목록), 버킷 정책, 액세스 지점 정책 또는 모두를 통해 버킷 및 객체에 부여됩니다. 이 버킷 및 해당 객체는 이 설정이 이 버킷 및 해당 액세스 지점에만 적용됩니다. AWS에서는 모든 퍼블릭 액세스 차단을 활성화하도록 권장하지만, 이 설정이 이 버킷 또는 내부 객체에 대한 어느 정도 수준의 퍼블릭 액세스가 필요한 경우 특정 스토리지 사용 사례에 맞게 아래 개별 설정을 사용하십시오.

☐ 모든 퍼블릭 액세스 차단

이 설정을 활성화하면 아래 4개의 설정을 모두 활성화한 것과 같습니다. 다음 설정 각각은 서로 독립적입니다.

☐ 새 ACL(액세스 제어 목록)을 통해 부여된 버킷 및 객체에 대한 퍼블릭 액세스 차단

S3은 새로 추가된 버킷 또는 객체에 적용되는 퍼블릭 액세스 권한을 차단하며, 기존 버킷 및 객체에 대한 새 퍼블릭 액세스 ACL 생성을 금지합니다.

☐ 임의의 ACL(액세스 제어 목록)을 통해 부여된 버킷 및 객체에 대한 퍼블릭 액세스 차단

S3은 버킷 및 객체에 대한 퍼블릭 액세스를 부여하는 모든 ACL을 무시합니다.

☐ 새 퍼블릭 버킷 또는 액세스 지점 정책을 통해 부여된 버킷 및 객체에 대한 퍼블릭 액세스 차단

S3은 버킷 및 객체에 대한 퍼블릭 액세스를 부여하는 새 버킷 및 액세스 지점 정책을 차단합니다. 이 설정은 S3 리소스에 대한 퍼블릭 액세스를 허용합니다.

☐ 임의의 퍼블릭 버킷 또는 액세스 지점 정책을 통해 부여된 버킷 및 객체에 대한 퍼블릭 및 교차 계정 액세스 차단

S3은 버킷 및 객체에 대한 퍼블릭 액세스를 부여하는 정책을 사용하는 버킷 또는 액세스 지점에 대한 퍼블릭 및 교차 계정 액세스를 무시합니다.

⚠ 모든 퍼블릭 액세스 차단을 비활성화하면 이 버킷과 그 안에 포함된 객체가 퍼블릭 상태가 될 수 있습니다.

정적 웹 사이트 호스팅과 같은 구체적으로 확인된 사용 사례에서 퍼블릭 액세스가 필요한 경우가 아니면 모든 퍼블릭 액세스 차단을 비활성화하지 마십시오.

☒ 현재 설정으로 인해 이 버킷과 그 안에 포함된 객체가 퍼블릭 상태가 될 수 있음을 알고 있습니다.

Step 2. Prod Bucket 생성

맨 아래 "버킷 만들기"를 클릭해 Prod용 버킷을 생성하고, 생성된 버킷을 확인합니다.

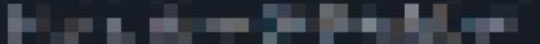
범용 버킷 (2) 정보

 ARN 복사 비어 있음 삭제 버킷 만들기

버킷은 S3에 저장되는 데이터의 컨테이너입니다.

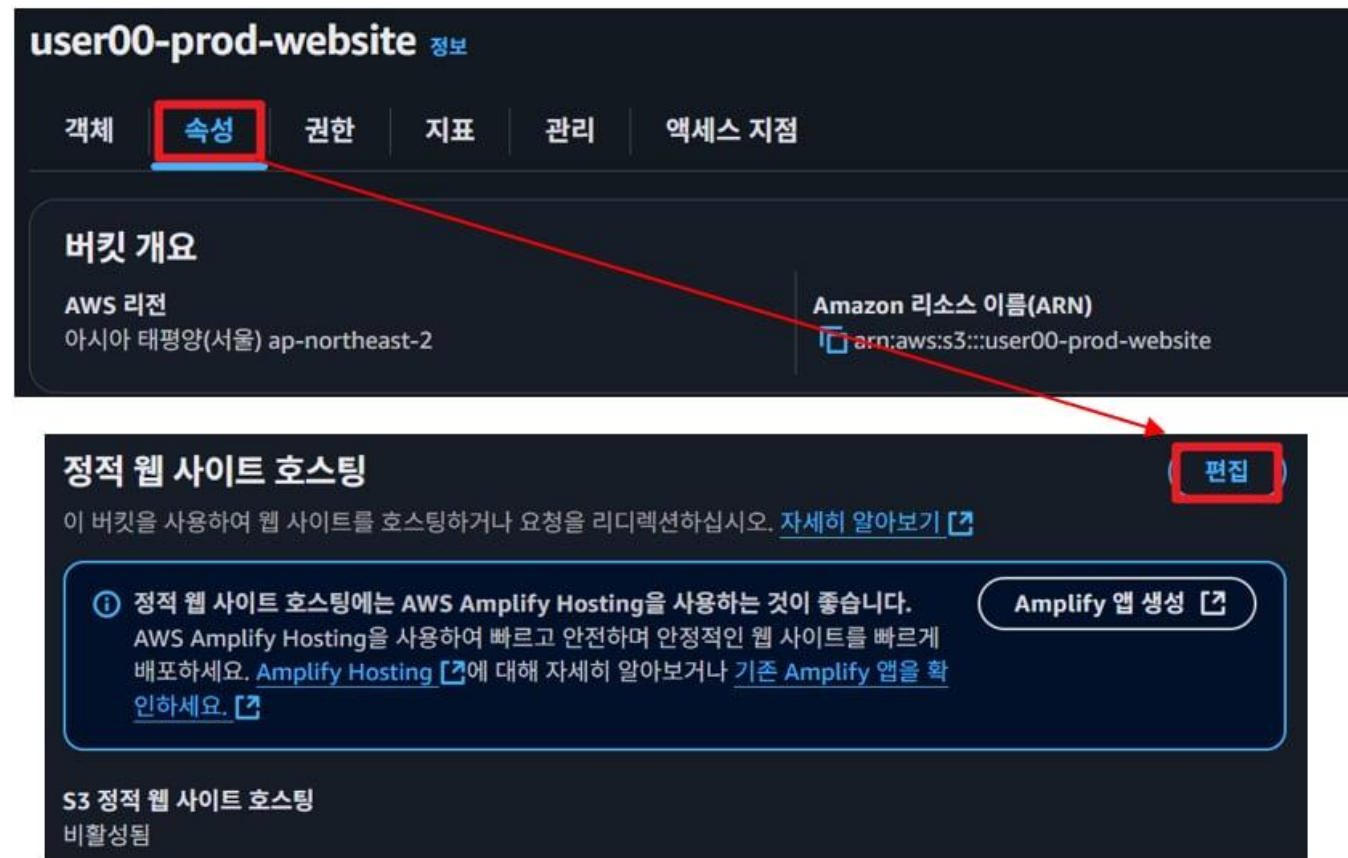
< 1 >



	이름 ▲	AWS 리전 ▼	생성 날짜 ▼
☐	user00-prod-website	아시아 태평양(서울) ap-northeast-2	
☐	user00-source-website	아시아 태평양(서울) ap-northeast-2	

Step 3. 정적 웹 사이트 호스팅

Prod 버킷에서 "속성"을 클릭하고, 아래쪽에 정적 웹 사이트 호스팅에서 "편집"을 클릭합니다.



user00-prod-website 정보

객체 속성 권한 지표 관리 액세스 지점

버킷 개요

AWS 리전
아시아 태평양(서울) ap-northeast-2

Amazon 리소스 이름(ARN)
arn:aws:s3:::user00-prod-website

정적 웹 사이트 호스팅

이 버킷을 사용하여 웹 사이트를 호스팅하거나 요청을 리디렉션하십시오. [자세히 알아보기](#)

정적 웹 사이트 호스팅에는 AWS Amplify Hosting을 사용하는 것이 좋습니다. AWS Amplify Hosting을 사용하여 빠르고 안전하며 안정적인 웹 사이트를 빠르게 배포하세요. [Amplify Hosting](#)에 대해 자세히 알아보거나 [기존 Amplify 앱을 확인하세요](#).

[Amplify 앱 생성](#)

S3 정적 웹 사이트 호스팅
비활성됨

Step 3. 정적 웹 사이트 호스팅

정적 웹 사이트 호스팅을 "활성화" 합니다.

정적 웹 사이트 호스팅

이 버킷을 사용하여 웹 사이트를 호스팅하거나 요청을 리디렉션하십시오. [자세히 알아보기](#)

정적 웹 사이트 호스팅

☐ 비활성화

☒ 활성화

인덱스 문서와 오류 문서를 아래와 같이 지정하고, "변경 사항 저장" 을 클릭합니다.

인덱스 문서

웹 사이트의 홈 페이지 또는 기본 페이지를 지정합니다.

index.html

오류 문서 - 선택 사항

오류가 발생하면 반환됩니다.

error.html

Step 4. 버킷 웹 사이트 엔드포인트 확인

Prod 버킷의 정적 웹 사이트 호스팅에서 "버킷 웹 사이트 엔드포인트"를 확인합니다.

정적 웹 사이트 호스팅

이 버킷을 사용하여 웹 사이트를 호스팅하거나 요청을 리디렉션하십시오. [자세히 알아보기](#)

① 정적 웹 사이트 호스팅에는 AWS Amplify Hosting을 사용하는 것이 좋습니다.

AWS Amplify Hosting을 사용하여 빠르고 안전하며 안정적인 웹 사이트를 빠르게 배포하세요. [Amplify Hosting](#)에 대해 자세히 알아보기

S3 정적 웹 사이트 호스팅

활성화됨

호스팅 유형

버킷 호스팅

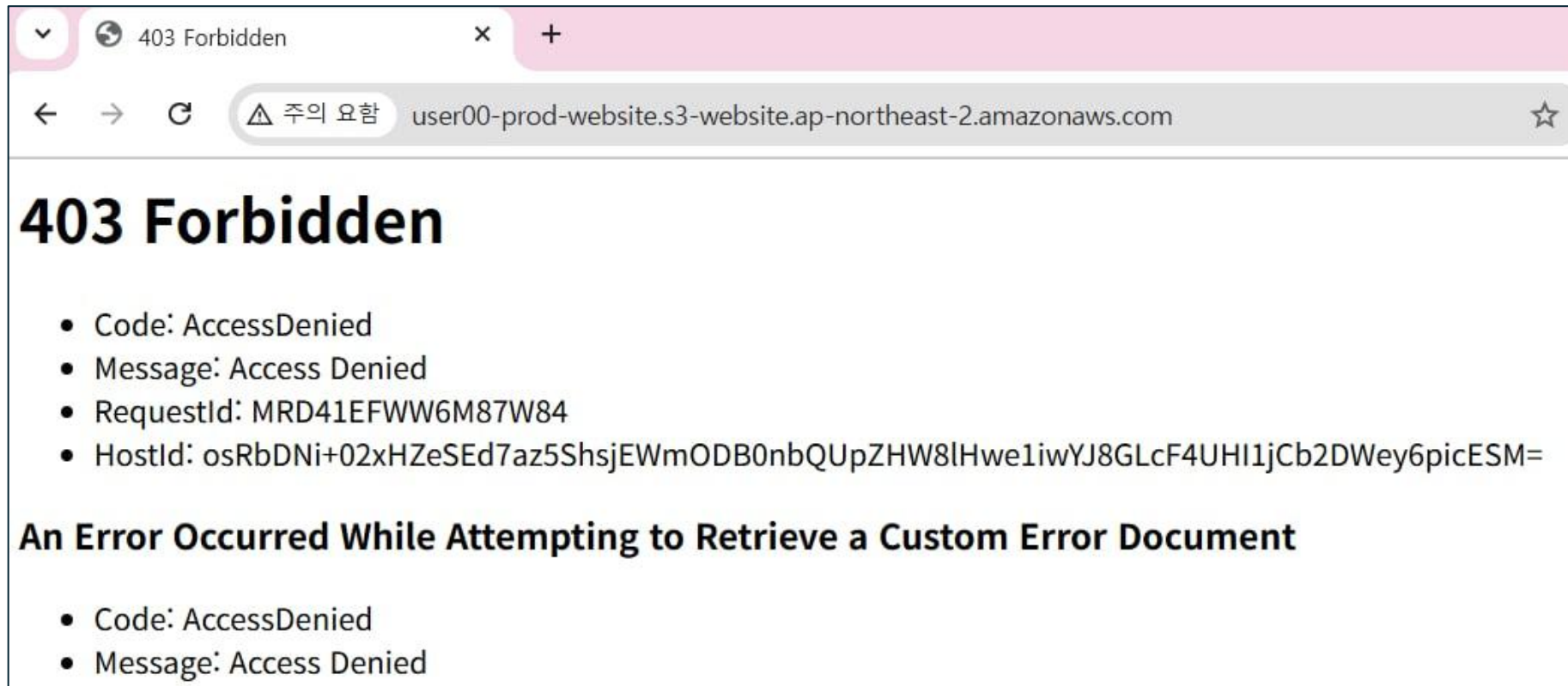
버킷 웹 사이트 엔드포인트

버킷을 정적 웹 사이트로 구성하면 해당 웹 사이트를 버킷의 AWS 리전별 웹 사이트 엔드포인트에서 사용할 수 있습니다. [자세히 알아보기](#)

<http://user00-prod-website.s3-website-ap-northeast-2.amazonaws.com>

Step 4. 버킷 웹 사이트 엔드포인트 확인

엔드포인트를 복사하고, 웹 브라우저에서 접속합니다.(접속 안됨)



Step 5. 웹 페이지 업로드

Source 버킷에 제공된 "test-website.zip" 파일을 업로드합니다.

user00-source-website 정보

객체

속성

권한

지표

관리

액세스 지점

객체 (1)

S3 URI 복사

URL 복사

다운로드

객체는 Amazon S3에 저장되어 있는 기본 엔터티입니다. [Amazon S3 인벤토리](#)를 사용하여 버킷에 있는 모든 객체의 목록을 얻을 수 있습니다. [자세히 알아보기](#)

Q 접두사로 객체 찾기

버전 표시

이름

▲

유형

▼

마지막 수정

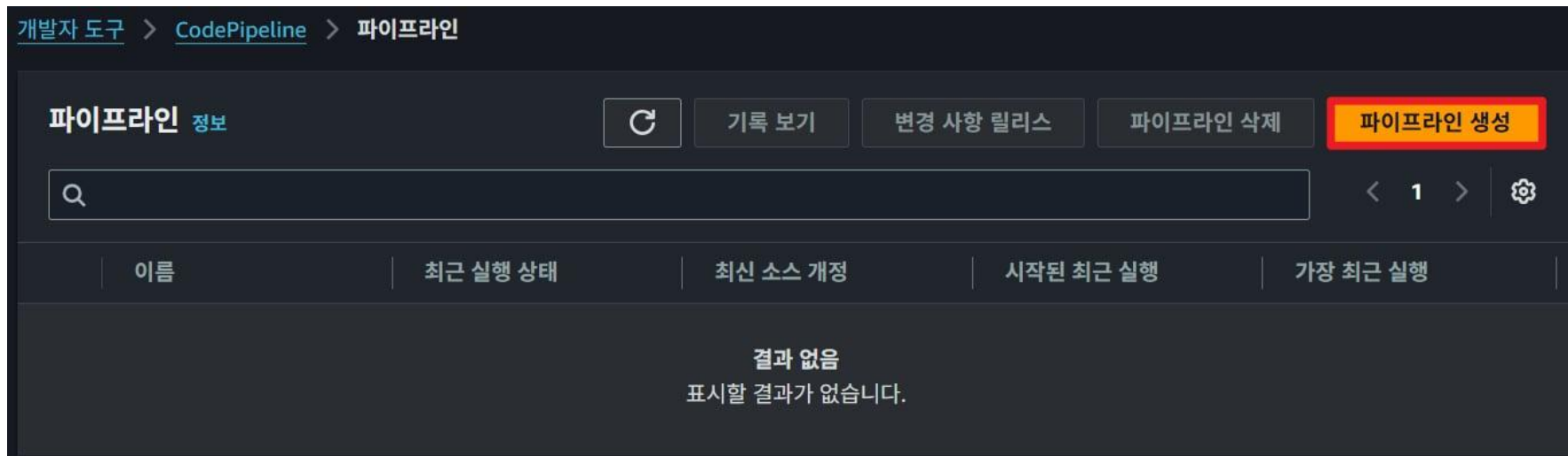
▼

test-website.zip

zip

Step 1. Pipeline 생성하기

AWS CodePipeline에서 "파이프라인 생성"을 클릭합니다.



Pipeline 생성

Step 1. Pipeline 생성하기

생성 옵션 선택에서 "사용자 지정 파이프라인 빌드"를 선택합니다.

파이프라인의 이름을 입력하고 다음을 선택합니다.

생성 옵션 선택 정보

Step 1 of 7

Category

☐ Deployment
 ☐ Continuous Integration
 ☐ Automation
 ☒ 사용자 지정 파이프라인 빌드

취소 다음

파이프라인 설정 선택 정보

Step 2 of 7

파이프라인 설정

파이프라인 이름

파이프라인 이름을 입력합니다. 생성된 후에는 파이프라인 이름을 편집할 수 없습니다.

user00-website-pipeline

100자를 초과할 수 없음

Step 1. Pipeline 생성하기

소스 스테이지에서 소스 공급자와 버킷, 객체키를 입력합니다.(객체키는 S3에 업로드한 파일명과 동일해야 합니다.)

소스 스테이지 추가 정보

Step 3 of 7

소스

소스 공급자

파이프라인의 입력 아티팩트를 저장한 위치입니다. 공급자를 선택한 후 연결 세부 정보를 제공하십시오.

Amazon S3

버킷

user00-source-website

S3 객체 키

test-website.zip

객체 키를 입력합니다. 시작 위치에 구분 기호 문자(/) 없이 파일 경로를 포함할 수 있습니다. 파일 확장명을 포함합니다(예: SampleApp.zip).

☒ EventBridge 규칙을 생성하여 자동으로 소스 변화를 감지합니다.

비활성화된 경우 AWS 설명서에 따라 소스에 대한 EventBridge 규칙을 생성하세요. [자세히 알아보십시오](#)

☒ 스테이지 장애 시 자동 재시도 활성화

취소

이전

다음

Step 1. Pipeline 생성하기

빌드 스테이지와 테스트 스테이지는 건너뛰니다.

빌드 스테이지 추가 정보

Step 4 of 7

빌드 - 선택 사항

빌드 공급자
빌드 명령을 실행하고 빌드 작업의 아티팩트를 지정하는 데 사용할 도구를 선택합니다.

☐ 명령

☒ 기타 빌드 공급자

☒ 스테이지 장애 시 자동 재시도 활성화

취소

이전

빌드 스테이지 건너뛰기

다음

Step 1. Pipeline 생성하기

배포 스테이지 추가에서 배포 공급자, 버킷을 지정하고, "배포하기 전에 파일 압축 풀기"를 체크합니다.

배포 스테이지 추가 정보

Step 6 of 7

이 스테이지는 건너뛸 수 없습니다.
파이프라인에 스테이지가 2개 이상 있어야 합니다. 두 번째 스테이지는 빌드, 테스트 또는 배포 스테이지여야 합니다. 빌드 스테이지, 테스트 스테이지 또는 배포 스테이지의 공급자를 선택하세요.

배포

배포 공급자
애플리케이션 또는 콘텐츠를 배포할 방법을 선택합니다. 공급자를 선택한 다음 해당 공급자에 대한 구성 세부 정보를 입력하세요.

Amazon S3

리전
아시아 태평양 (서울)

입력 아티팩트
이 작업의 입력 아티팩트를 선택합니다. 자세히 알아보십시오

SourceArtifact
Source에 의해 정의됨
100자 이하

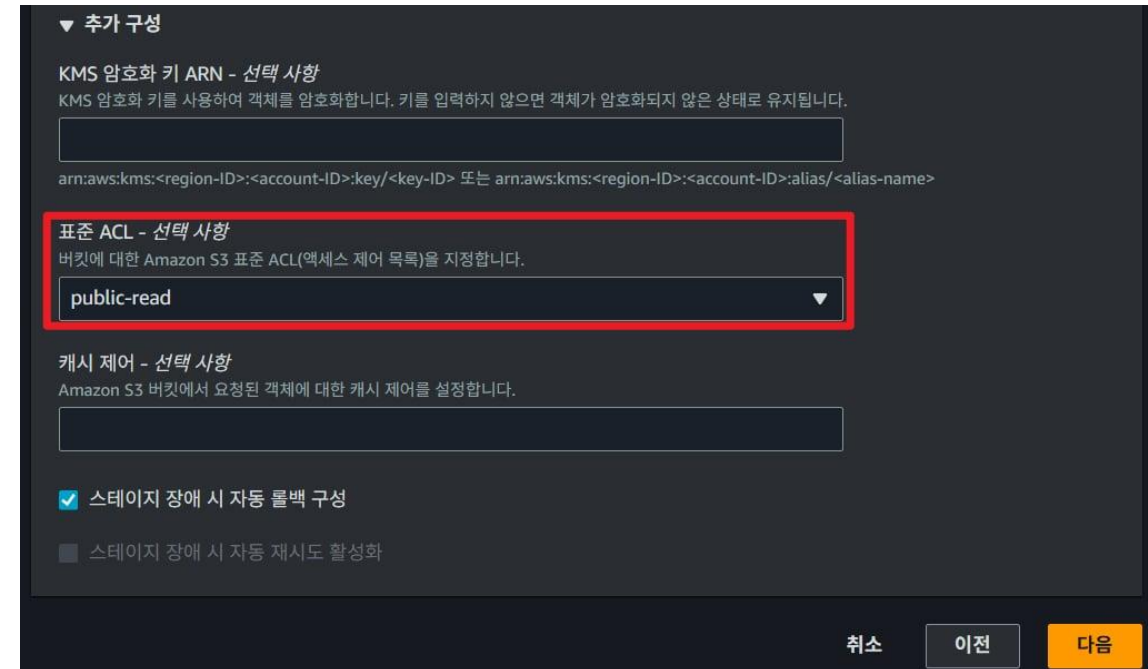
버킷
user00-prod-website

배포 경로 - 선택 사항

☒ 배포하기 전에 파일 압축 풀기
아티팩트를 배포하기 전에 아티팩트의 압축을 풉니다.

Step 1. Pipeline 생성하기

추가 구성을 확장해 표준 ACL을 "public-read"로 지정하고, 다음을 클릭합니다.



▼ 추가 구성

KMS 암호화 키 ARN - 선택 사항
KMS 암호화 키를 사용하여 객체를 암호화합니다. 키를 입력하지 않으면 객체가 암호화되지 않은 상태로 유지됩니다.

arn:aws:kms:<region-ID>:<account-ID>:key/<key-ID> 또는 arn:aws:kms:<region-ID>:<account-ID>:alias/<alias-name>

표준 ACL - 선택 사항
버킷에 대한 Amazon S3 표준 ACL(엑세스 제어 목록)을 지정합니다.

캐시 제어 - 선택 사항
Amazon S3 버킷에서 요청된 객체에 대한 캐시 제어를 설정합니다.

☒ 스테이지 장애 시 자동 롤백 구성

☐ 스테이지 장애 시 자동 재시도 활성화

취소 이전 다음

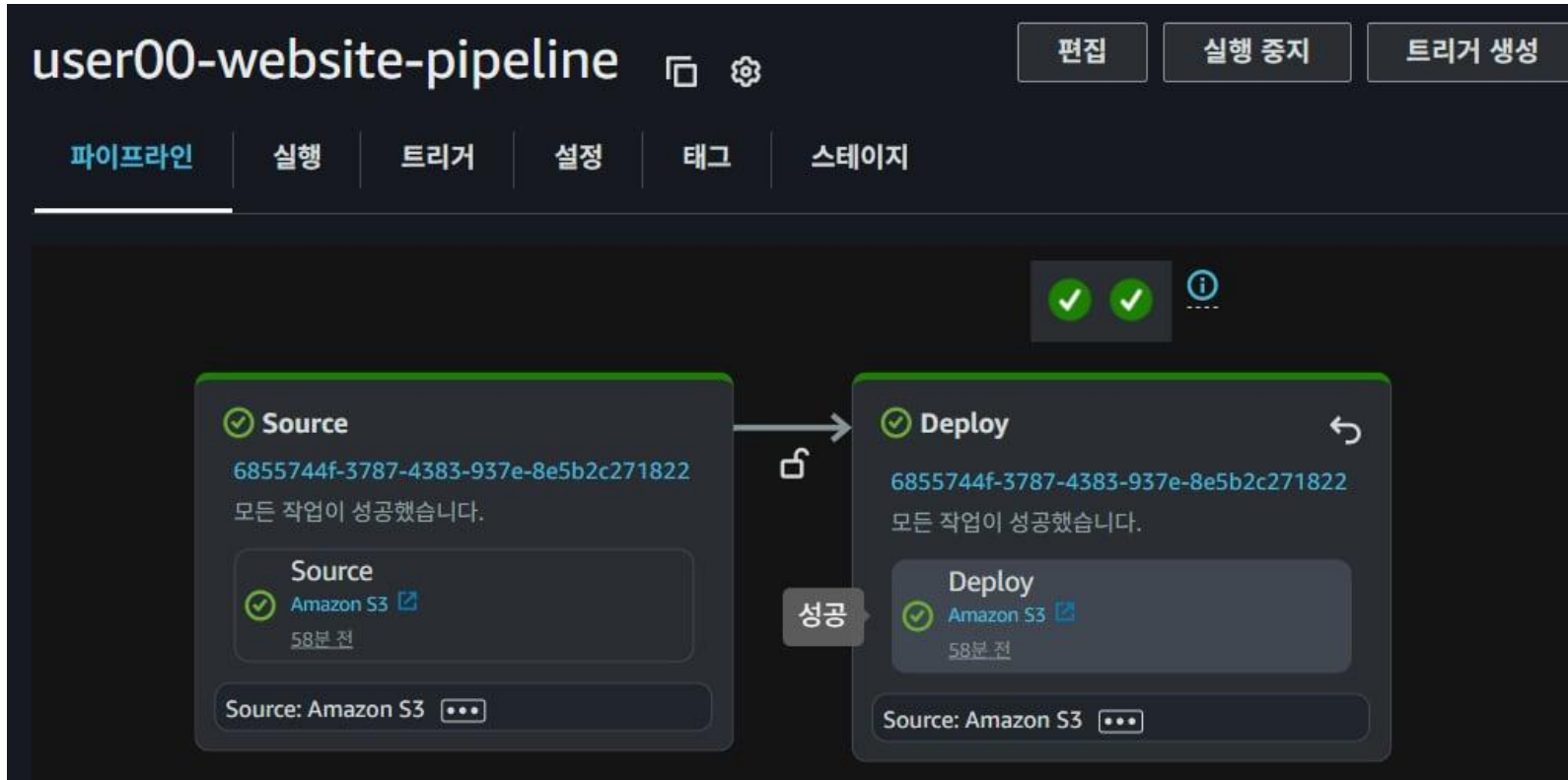
검토 단계에서 "파이프라인 생성"을 클릭합니다.



취소 이전 **파이프라인 생성**

Step 2. 파이프라인 확인

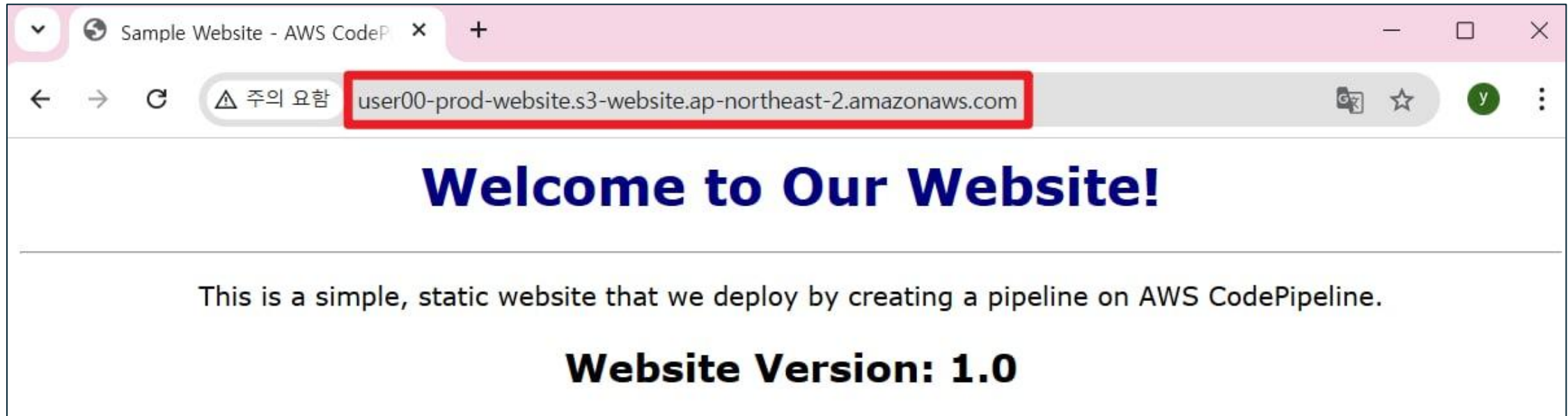
파이프라인 인터페이스에서 Source와 Deploy 단계가 성공했음을 확인합니다.(1~2분 소요)



The screenshot displays the 'user00-website-pipeline' interface. At the top, there are tabs for '파이프라인' (Pipeline), '실행' (Execution), '트리거' (Trigger), '설정' (Settings), '태그' (Tags), and '스테이지' (Stages). The '파이프라인' tab is selected. Below the tabs, there are three buttons: '편집' (Edit), '실행 중지' (Stop Execution), and '트리거 생성' (Create Trigger). The main area shows two stages: 'Source' and 'Deploy'. Both stages are marked with a green checkmark and the text '모든 작업이 성공했습니다.' (All jobs succeeded). The 'Source' stage is connected to the 'Deploy' stage by an arrow. A '성공' (Success) label is placed between the two stages. Each stage has a sub-section with a green checkmark and the text 'Source' or 'Deploy', followed by 'Amazon S3' and '58분 전' (58 minutes ago). The 'Source' stage also has a 'Source: Amazon S3' label at the bottom.

Step 3. 웹 사이트 접속 확인

웹 사이트에 정상적으로 접속이 되는지 확인합니다.



Step 1. 전환 비활성화

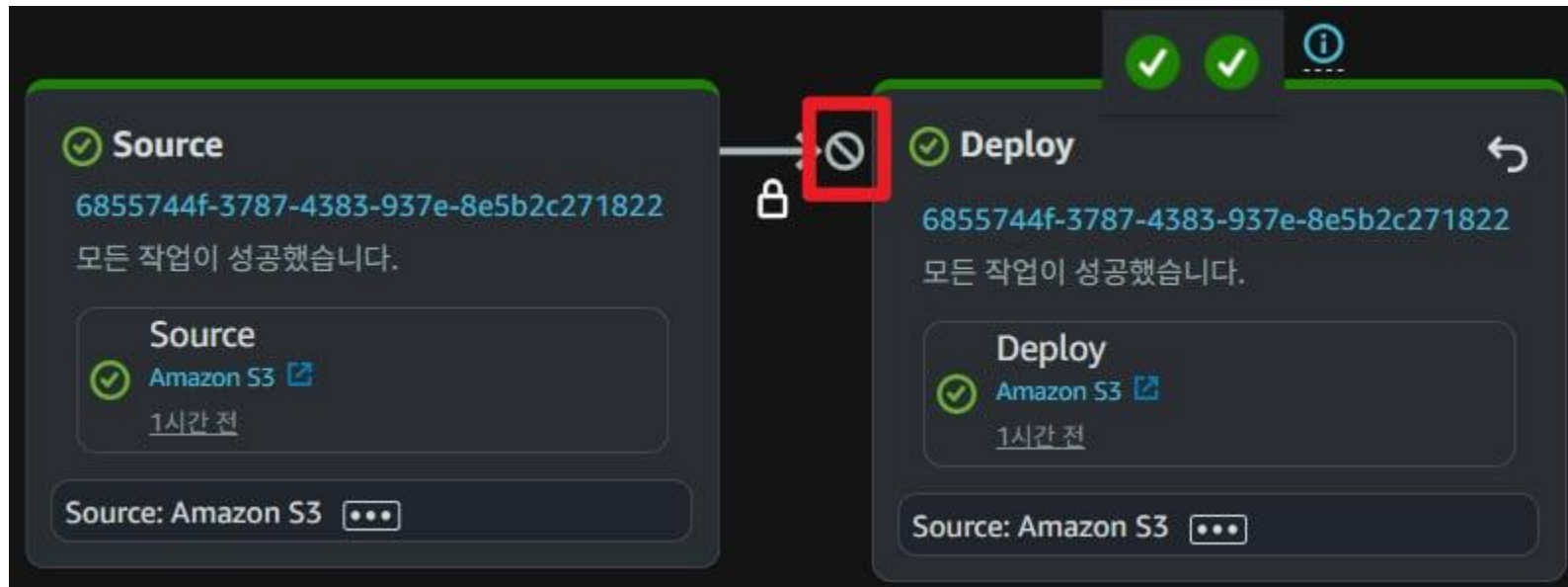
전환을 비활성화 합니다. 전환 비활성화 시 이유 (e.g., for demonstration)를 작성해야 합니다.



웹 사이트 업데이트

Step 1. 전환 비활성화

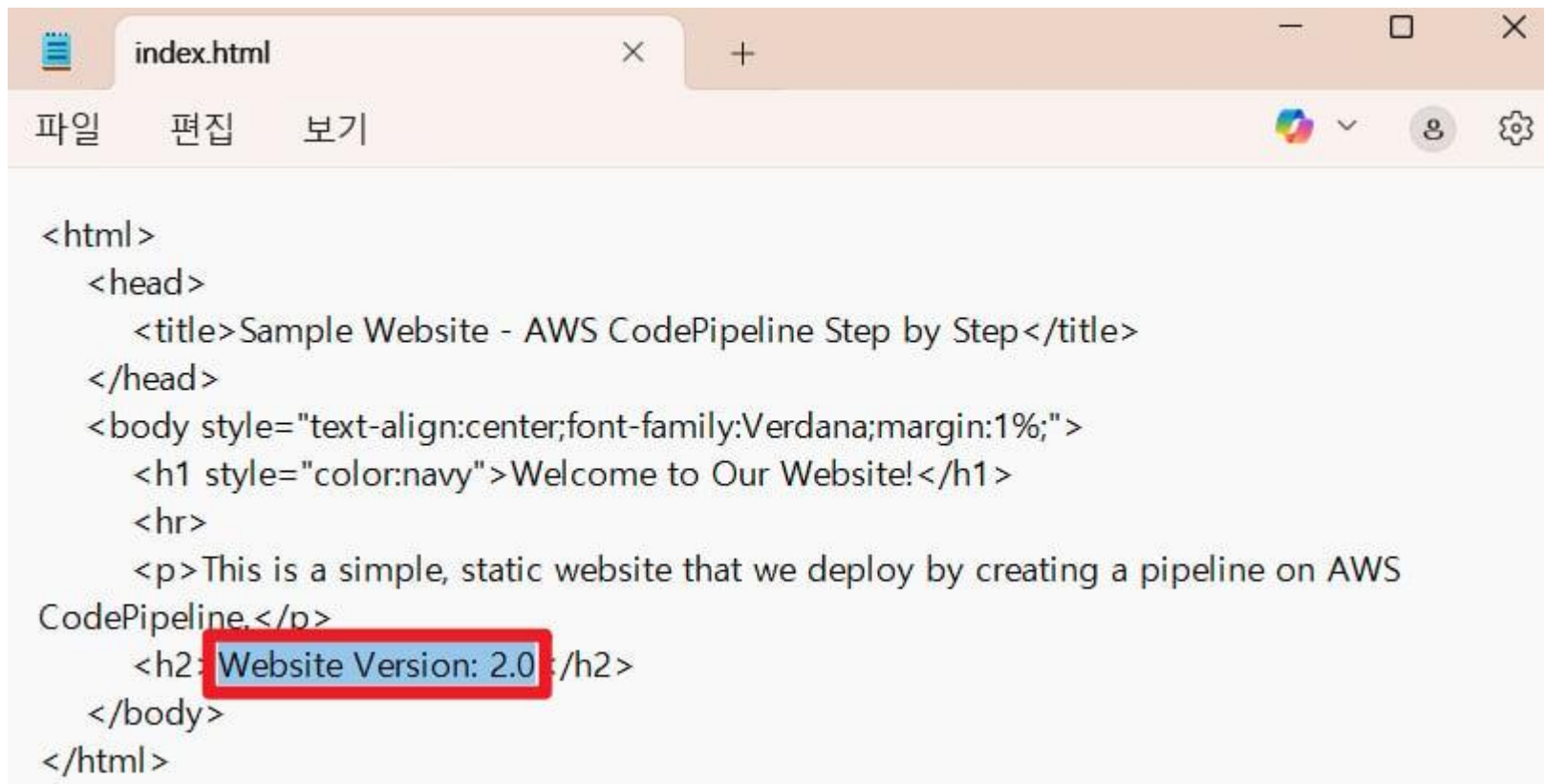
전환 **비활성화** 되었음을 확인합니다.



웹 사이트 업데이트

Step 2. 소스 파일 수정

제공된 test-website.zip을 압축 해제하고, index.html의 내용을 수정합니다.



```
<html>
  <head>
    <title>Sample Website - AWS CodePipeline Step by Step</title>
  </head>
  <body style="text-align:center;font-family:Verdana;margin:1%;">
    <h1 style="color:navy">Welcome to Our Website!</h1>
    <hr>
    <p>This is a simple, static website that we deploy by creating a pipeline on AWS
CodePipeline.</p>
    <h2>Website Version: 2.0</h2>
  </body>
</html>
```

웹 사이트 업데이트

Step 2. 소스 파일 수정

수정된 index.html과 error.html을 다시 압축합니다.



웹 사이트 업데이트

Step 3. 수정된 파일 업로드

S3 버킷의 Source 버킷에 수정된 파일을 업로드합니다.

user00-source-website

정보

객체

속성

권한

지표

관리

액세스 지점

객체 (1)

↺

S3 URI 복사

URL 복사

객체는 Amazon S3에 저장되어 있는 기본 엔터티입니다. [Amazon S3 인벤토리](#)를 사용하여 버킷에 있는 모든 객체의 목록을 얻을 수 있습니다.

🔍 접두사로 객체 찾기

버전 표시

☐	이름	▲	유형	▼	마지막 수정	▼	크기	▼	스토리지 클래스
☐	📄 test-website.zip		zip				782.0B		Standard

웹 사이트 업데이트

Step 3. 수정된 파일 업로드

버킷에 업로드 된 객체는 동일한 객체를 버전으로 저장하는 기능이 있습니다. 버전 표시 기능을 이용해 버전을 확인할 수 있습니다.

user00-source-website 정보

객체

속성

권한

지표

관리

액세스 지점

객체 (2)

🔄

S3 URI 복사

URL 복사

객체는 Amazon S3에 저장되어 있는 기본 엔터티입니다. [Amazon S3 인벤토리](#)를 사용하여 버킷에 있는 모든 객체의 목록을 얻을 수 있습니다.

🔍 접두사로 객체 찾기

☒ 버전 표시

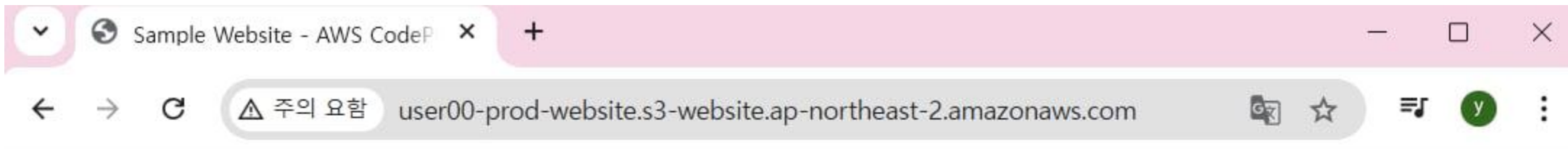
☐	이름	▲	유형	버전 ID	마지막 수정
☐	test-website.zip		zip	fuQetSHfUB3.gFLhgG.VOP_nugjPnw5z	
☐	test-website.zip		zip	5Aq0W0DegHk7locudx.ZYtgDNdZrmLD3b	

KT AIVLE School

50

Step 3. 수정된 파일 업로드

웹 사이트를 확인합니다. 현재는 파이프라인의 전환이 비활성화 되어 있으므로 업데이트 되지 않은 웹 사이트가 나옵니다.



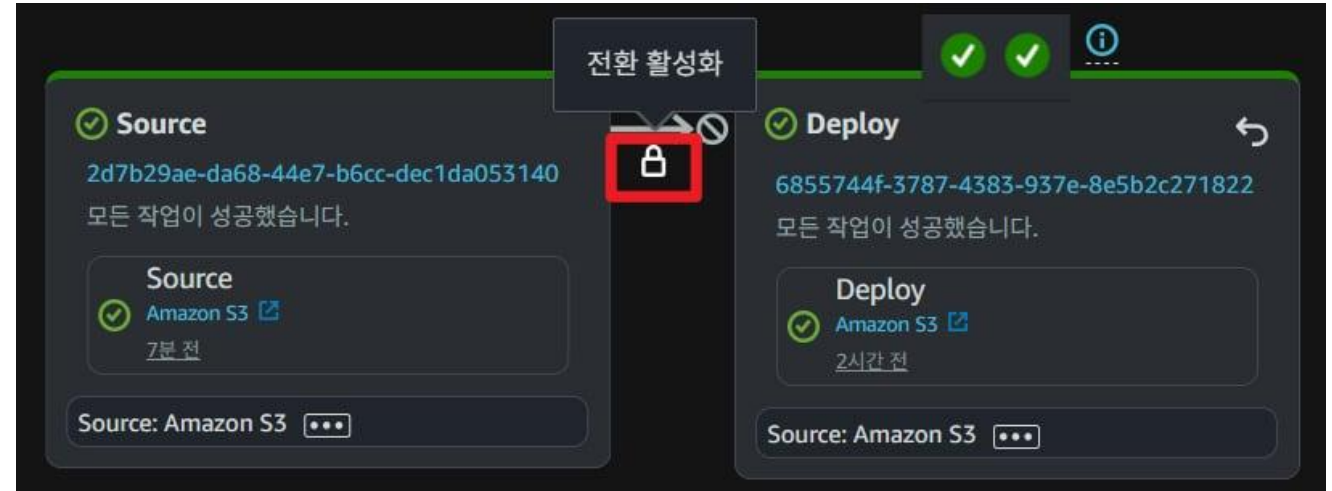
Welcome to Our Website!

This is a simple, static website that we deploy by creating a pipeline on AWS CodePipeline.

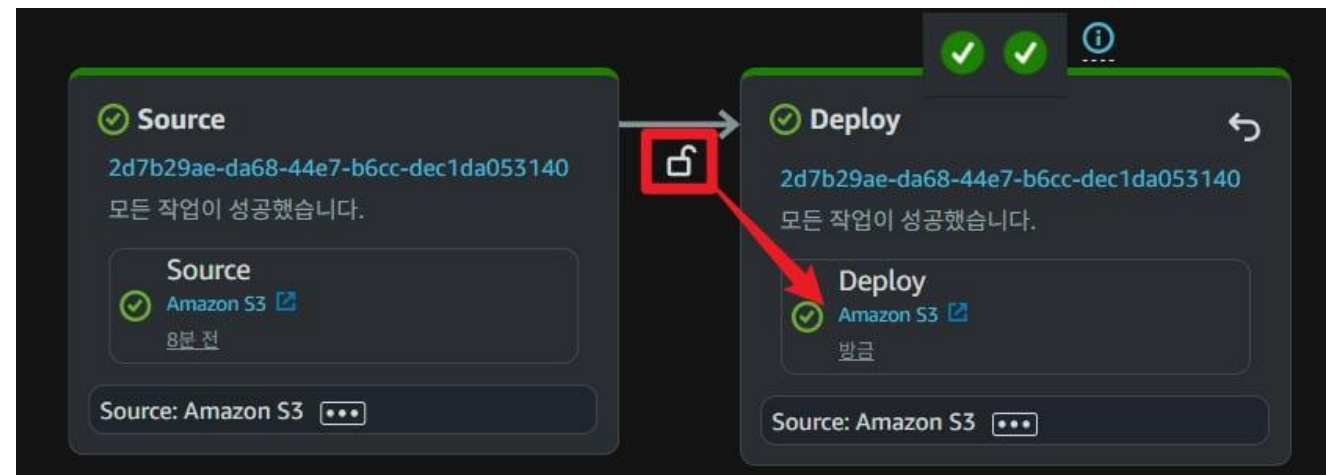
Website Version: 1.0

웹 사이트 업데이트

Step 4. 파이프라인 전환 활성화
CodePipeline에서 전환을 활성화합니다.

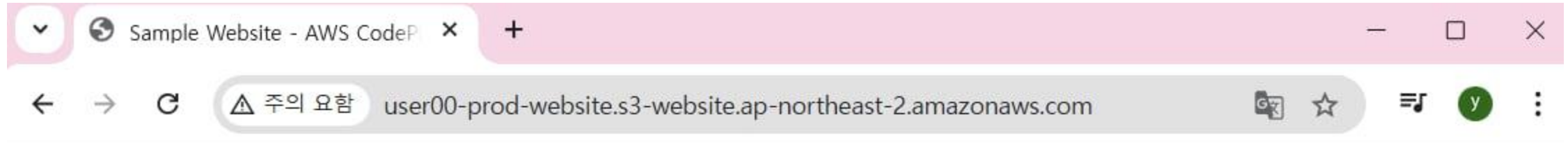


배포가 정상적으로 성공 됨을 확인합니다.



Step 4. 파이프라인 전환 활성화

웹브라우저에서 웹사이트가 업데이트 되었음을 확인합니다.



Welcome to Our Website!

This is a simple, static website that we deploy by creating a pipeline on AWS CodePipeline.

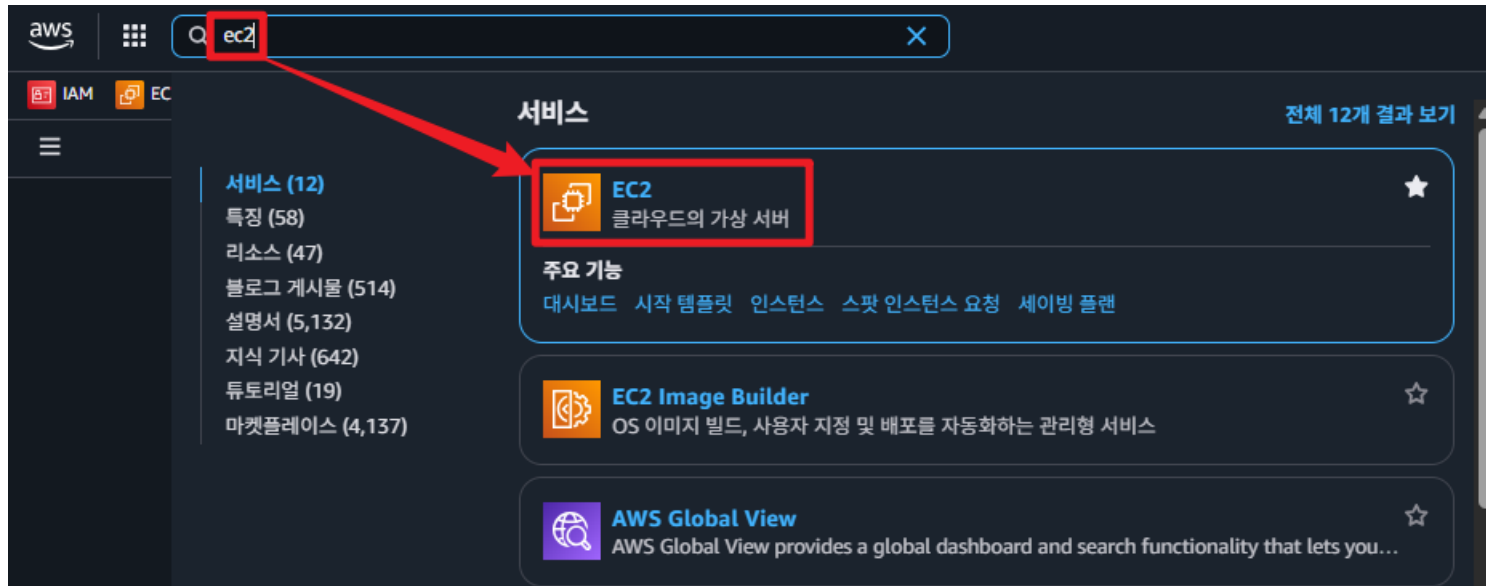
Website Version: 2.0

Lab 03.

Amazon EC2 Instance 연결 및 생성

Step 1. 인스턴스 시작

AWS Management Console에서 EC2를 검색합니다.



Instance 생성

Step 1. 인스턴스 시작

EC2 대시보드에서 '인스턴스 시작'을 클릭합니다.

EC2

대시보드

EC2 글로벌 보기

이벤트

인스턴스

인스턴스

인스턴스 유형

시작 템플릿

스팟 요청

Savings Plans

예약 인스턴스

전용 호스트

용량 예약

Capacity Manager

이미지

리소스

아시아 태평양 (서울) 리전에서 다음 Amazon EC2 리소스를 사용하고 있음:

인스턴스(실행 중)	0	로드 밸런서
보안 그룹	3	블록
인스턴스	5	전용 호스트
탄력적 IP	0	Auto Scaling 그룹

인스턴스 시작

시작하려면 클라우드의 가상 서버인 Amazon EC2 인스턴스를 시작하십시오.

인스턴스 시작

서버 마이그레이션

참고: 인스턴스는 아시아 태평양 (서울) 리전에서 시작됩니다.

Instance 생성

Step 2. 이름 및 태그

생성할 인스턴스의 이름을 작성합니다.

형식: **user##-test-ec2-instance**

인스턴스 시작 정보

Amazon EC2를 사용하면 AWS 클라우드에서 실행되는 가상 머신 또는 인스턴스를 생성할 수 있습니다. 아래의 간단한 단계에 따라 빠르게 시작할 수 있습니다.

이름 및 태그 정보

이름

추가 태그 추가

Step 2. 애플리케이션 및 OS 이미지(Amazon Machine Image) 생성할 Instance의 이미지를 선택(Ubuntu)합니다.

▼ 애플리케이션 및 OS 이미지(Amazon Machine Image) 정보

AMI에는 인스턴스의 운영 체제, 애플리케이션 서버, 애플리케이션이 포함됩니다. 아래에 적합한 AMI가 보이지 않는 경우 검색 필드를 사용하거나 더 많은 AMI 찾아보기를 선택하세요.

Q 추천 개의 애플리케이션 및 OS 이미지를 포함하는 전체 카탈로그 검색

최근 사용

Quick Start

Amazon Linux

macOS

Ubuntu

Windows

Red Hat

SUSE Linux

Debian

aws

Mac

ubuntu

Microsoft

Red Hat

debian

더 많은 AMI 찾아보기

AWS, Marketplace 및 커뮤니티의 AMI 포함

Amazon Machine Image(AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
ami-0a71e3eb8b23101ed (64비트(x86)) / ami-066f9893a857529ea (64비트(Arm))
가상화: hvm ENA 활성화됨: true 루트 디바이스 유형: ebs

프리 티어 사용 가능

설명

Ubuntu Server 24.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Canonical, Ubuntu, 24.04, amd64 noble image

아키텍처

AMI ID

게시 날짜

사용자 이름

64비트(x86)

ami-0a71e3eb8b23101ed

2025-10-22

ubuntu

확인된 공급 업체

Step 3. 인스턴스 유형

인스턴스 유형을 선택(t3.micro)합니다.

▼ 인스턴스 유형 [정보](#) | [조언 받기](#)

인스턴스 유형

t3.micro

패밀리: t3 2 vCPU 1 GiB 메모리 현재 세대: true 온디맨드 Windows 기본 요금: 0.0222 USD 시간당
온디맨드 Linux 기본 요금: 0.013 USD 시간당 온디맨드 RHEL 기본 요금: 0.0418 USD 시간당
온디맨드 SUSE 기본 요금: 0.013 USD 시간당 온디맨드 Ubuntu Pro 기본 요금: 0.0165 USD 시간당

프리 티어 사용 가능 ▼

☐ 모든 세대

[인스턴스 유형 비교](#)

소프트웨어가 사전 설치된 AMI에는 추가 비용이 적용됩니다.

Step 4. 키 페어(로그인)

“새 키 페어를 생성”(기존 설정 유지)을 클릭하고, 새로운 키 페어를 만듭니다. **생성된 키는 로컬에 잘 저장해 둡니다.(잊어버리면 안됨)**

이후 생성한 키를 선택합니다.

▼ 키 페어(로그인) 정보

키 페어를 사용하여 인스턴스에 안전하게 연결할 수 있습니다. 인스턴스를 시작하기 전에 선택한 키 페어에 대한 액세스 권한이 있는지 확인하세요.

키 페어 이름 - 필수

▼

 새 키 페어 생성

Step 5. 네트워크 설정

VPC는 'Default VPC'를 선택하고, 퍼블릭 IP 자동 할당을 '활성화'로 선택합니다. 방화벽은 '기존 보안 그룹 선택'을 클릭합니다. 이후 오른쪽 하단에 '인스턴스 시작'을 클릭해 인스턴스를 생성합니다.

▼ 네트워크 설정 정보

VPC - 필수 정보

vpc-07992fd8f6e27fe02 (기본값) ↕
172.31.0.0/16

서브넷 정보

기본 설정 없음 ↕ 새 서브넷 생성 ↗

가용 영역 정보

기본 설정 없음 ↕ 추가 영역 활성화 ↗

퍼블릭 IP 자동 할당 정보

활성화 ↕

프리 티어 허용 범위를 벗어나는 경우 추가 요금이 적용됩니다.

방화벽(보안 그룹) 정보

보안 그룹은 인스턴스에 대한 트래픽을 제어하는 방화벽 규칙 세트입니다. 특정 트래픽이 인스턴스에 도달하도록 허용하는 규칙을 추가합니다.

☐ 보안 그룹 생성

☒ 기존 보안 그룹 선택

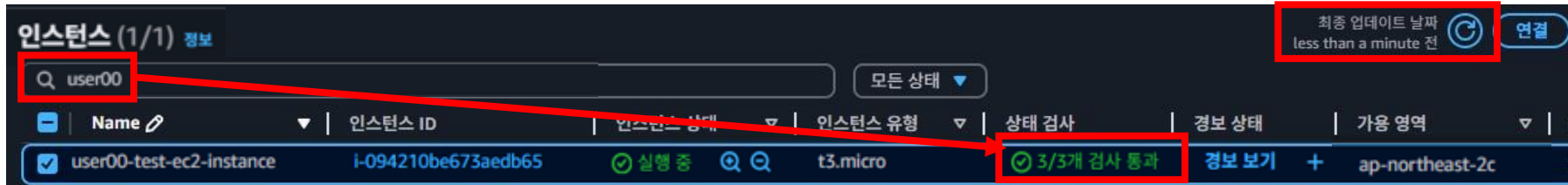
일반 보안 그룹 정보

보안 그룹 선택 ↕ 보안 그룹 규칙 비교

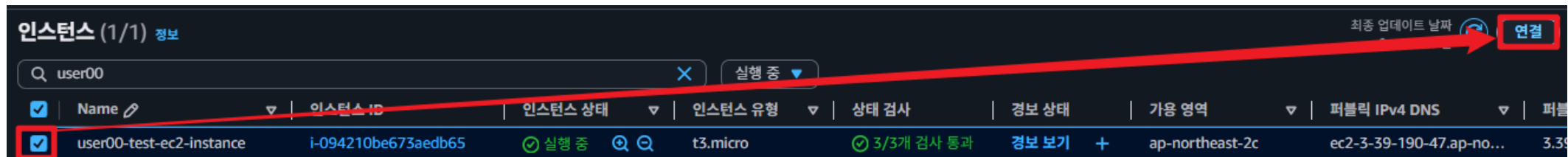
여기서 추가 또는 제거하는 보안 그룹은 모든 네트워크 인터페이스에서 추가 또는 제거됩니다.

Step 6. 생성된 인스턴스 확인

필터 기능을 이용해 생성한 인스턴스를 확인합니다. 새로 고침 후, 상태 검사에 '검사 통과'를 확인합니다.



인스턴스를 선택하고, '연결'을 클릭합니다.



Step 7. SSH 연결

SSH 클라이언트 탭에서 연결하기 위한 예를 복사합니다.

연결 정보
브라우저 기반 클라이언트를 사용하여 인스턴스에 연결합니다.

EC2 인스턴스 연결 | Session Manager | **SSH 클라이언트** | EC2 직렬 콘솔

인스턴스 ID
i-094210be673aedb65 (user00-test-ec2-instance)

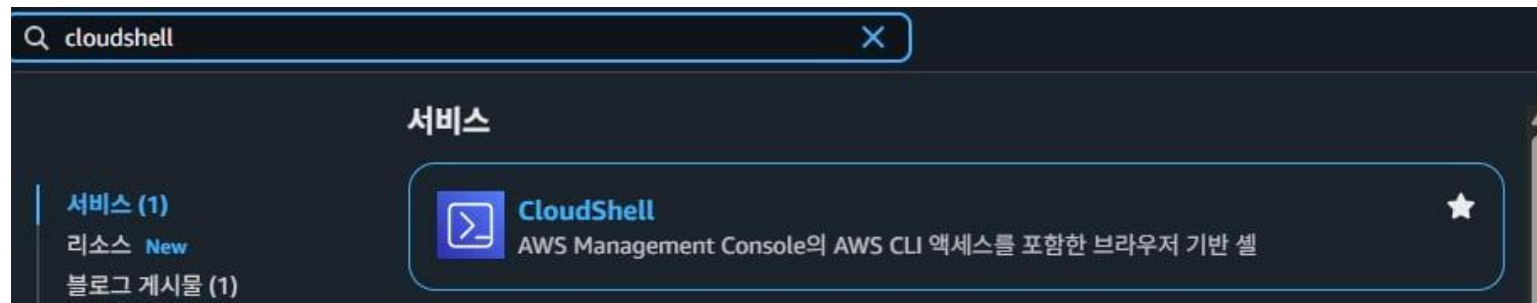
1. SSH 클라이언트를 엽니다.
2. 프라이빗 키 파일을 찾습니다. 이 인스턴스를 시작하는 데 사용되는 키는 user00-keypair.pem입니다.
3. 필요한 경우 이 명령을 실행하여 키를 공개적으로 볼 수 없도록 합니다.
`chmod 400 "user00-keypair.pem"`
4. 퍼블릭 DNS 이름(들) 사용하여 인스턴스에 연결:
`ec2-3-39-190-47.ap-northeast-2.compute.amazonaws.com`

예:
`ssh -i "user00-keypair.pem" ubuntu@ec2-3-39-190-47.ap-northeast-2.compute.amazonaws.com`

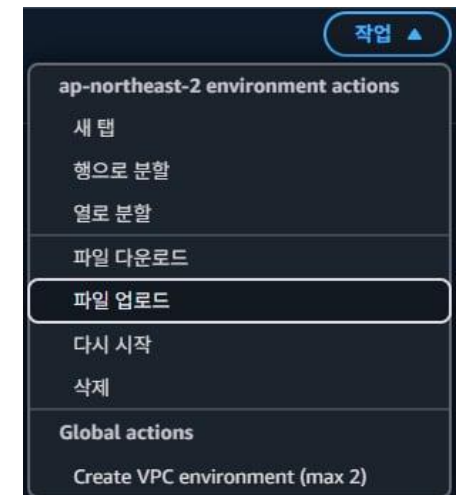
① 참고: 대부분의 경우 추정된 사용자 이름은 정확합니다. 하지만 AMI 사용 지침을 읽고 AMI 소유자가 기본 AMI 사용자 이름을 변경했는지 확인하세요.

Step 7. SSH 연결

AWS 서비스 검색 창에 'CloudShell'을 검색하고, 선택합니다.



작업 > 파일 업로드를 클릭해 Instance를 만들 때 생성한 keypair를 업로드 합니다.



Step 7. SSH 연결

.pem이 업로드 되었는지 확인합니다.

```
ls
```

CloudShell

ap-northeast-2 x +

```
~ $ ls  
user00-keypair.pem
```

pem key의 권한을 설정합니다.

```
chmod 400 "user##-keypair.pem"
```

Step 7. SSH 연결

SSH 클라이언트 탭에서 복사한 명령어로, 퍼블릭 DNS를 사용하여 Instance에 연결합니다.
(ec2-x-x-x-x에서 x는 Instance의 Public IP로 작성합니다)

```
ssh -i "user##-keypair.pem" ubuntu@ec2-x-x-x-x.ap-northeast-2.compute.amazonaws.com
```

Are you sure you want to continue connecting (yes/no/[fingerprint])? 에서 yes를 입력합니다.

```
~ $ ssh -i "user00-keypair.pem" ubuntu@ec2-3-38-112-136.ap-northeast-2.compute.amazonaws.com
The authenticity of host 'ec2-3-38-112-136.ap-northeast-2.compute.amazonaws.com (3.38.112.136)' can't be established.
ED25519 key fingerprint is SHA256:JGV06n29CWIIzf/Edc8vJx6SQUN5a8bWj+4euudcqU.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

